

Chapter 4: Results: Designing an "Agentic BI" Architecture

4.1 Extraction of Source Patterns (MetaGPT)

This first phase, is about 'dissecting' the source domain, which is the *MetaGPT* architecture. In their research, Hong et al. (2024) demonstrate that complex coding, software engineering and system-building tasks are solved much more effectively when handled by a team of specialized AI agents. An important precision to make is that *MetaGPT* is not just a hardcoded application, but rather a multi-agent collaborative framework. To prove its effectiveness the authors instantiated this framework with their "First AI Software Company" agentic system case. The framework was thus initially conceptualized and tested for use within a software engineering context, but they mention that "you can use this multi-agent framework in your own scenario to build your own application", and even provide multiple tutorials, additional use case agents (e.g. Data Interpreter Agent), etc. to use and build own new application scenarios (Deepwisdom, n.d.). Their developed "Software Company" agentic system basically mimics the functioning of a real-life software development company, by assigning distinct human-like roles to the agents.

Through the extraction process, 4 main mechanisms from the *MetaGPT* framework were identified as essential to enable governed multi-agent collaboration. These four core components will serve as a 'blueprint' for the transposition of the architecture into the BI domain. They are the following:

1. **Role Specialization:** The assignment of distinct, human-like personas (with each specific skills, goals, and constraints) to individual LLM-powered agents in the system.
2. **Standard Operating Procedures (SOPs):** Predefined, strict, sequential workflows and guidelines that tell agents exactly which tasks they must do, and how and when to perform these tasks.
3. **Inter-agent Communication & Memory Mechanisms:** A 'publish-subscribe' shared message pool for communication between agents, to ensure that each of them works based on the correct, structured information it needs with no unnecessary noise information.
4. **Iterative Feedback and Self-Correction Mechanism:** Built-in review mechanisms that allow agents to check, debug and refine their outputs based on auto-

ated testing, and to record debug experiences in their memory to improve future performance.

The following sections provide more details about each of these 4 components, by exploring how they function within the original Software Company system of *MetaGPT*.

4.1.1 Role Specialization

In the same way that a real-world software development process involves various complex tasks for which the collaboration of different people with different expertise is needed, this is also how the agent collaboration is structured in *MetaGPT*. A specific role with specific skills and expertise is specified and assigned to each agent. Each of them has a specific profile or persona, name, goal, set of constraints, a set of executable actions and/or tools and a specific context. An example of these attributes for the Product Manager Agent is visible in Figure 7. All of these agents behave based on the ReAct (Reasoning & Acting) paradigm introduced by Yao et al. (2022), which means that they follow an iterative loop of "Thought $\rightarrow$ Action $\rightarrow$ Observation". They reason plan an action, then execute it, then observe its result, and use this observation to inform the next 'Thought'.

From an implementation-directed point of view, this role mechanism is realized in *MetaGPT* based on two main base classes the 'standard' `Role` class and the more extended `RoleZero` class. This second one, is a specialization of the `Role` class and thus inherits and extends its working. Both support the ReAct paradigm (with a `_think()` and a `_act()` method), but they differ in how this loop is implemented. In the `Role` class, the ReAct loop is executed with a `max_react_loop` of 1 by default, which means that the agent performs only one cycle and then stops. To determine the agent's next action, three execution options are possible: 'REACT' (the LLM is shown a list of pre-registered actions, thinks and chooses one of these), 'BY_ORDER' (actions are executed sequentially in a fixed, defined order), or 'PLAN_AND_ACT' (the LLM first constructs a plan of multiple sequential actions based on the shown list, then executes it).

In contrast to this, in the `RoleZero` class only the 'REACT' mode can be used and the `max_react_loop` is set to 50 by default. This enables longer autonomous reasoning loops. Instead of selecting from a fixed action list, the `_think()` method assembles a prompt (containing role instructions, available tool schemas, plan status and recent memory) and calls the LLM to decide based on this which tool or command to use next. Additionally, unlike `Role`, `RoleZero` re-observes the Environment at every iteration, to see if there is new information to adapt to mid-task. It also includes a human-in-the-loop fallback possibility for when the limit of actions is reached. Another extra method it possesses is `_quick_think()`, which does a pre-classification of the request into "QUICK", "SEARCH" or "TASK" before entering the ReAct loop, in order to use a more lightweight response path when the full ReAct cycle is not needed. Finally, `RoleZero` also supports optional long-term memory, which is not present in the `Role` class.

Lastly, it is also worth mentioning that the `RoleZero` class possesses a `use_fixed_sop` attribute, that, when set to True, makes the Object fall back to the standard inherited `Role` behavior. This thus makes both 'modes' available within one same role definition,

and switchable according to programmer needs.

To illustrate this, in the Software Company system, the Role Specialization mechanism is used to define the following roles :

- **Product Manager:** Analyzes requirements and writes the Product Requirement Document (PRD).
- **Architect:** Translates the PRD into technical system designs and API structures.
- **Project Manager:** Breaks down the architecture into sequential coding tasks.
- **Engineer:** Writes the actual Python/software code based on the tasks.
- **QA Engineer:** Reviews the code, writes tests, and sends it back if it fails.

4.1.2 Standard Operating Procedures

To prevent hallucinations, and unstructured collaboration between the different agents, the *MetaGPT* framework uses Standard Operating Procedures (SOPs) so that all agents work in a sequential, rigid way, enabling them to follow a strict and streamlined workflow. The authors define SOPs as "the established procedures that govern agent behaviors and interactions, ensuring orderly and efficient operations within the system." (Deepwisdom, n.d.). In essence, these SOPs are the strict definitions of which tasks each agent must perform, in which order, and with which inputs and outputs, received from and delivered to which agent. To make the agents adhere to it, they are encoded into the prompts that define the instructions for each role. Figure 6 illustrates how real-world human team software development SOPs are also used to structure the *MetaGPT* agent collaboration. As can be seen on the figure, in the Software Company system, the general SOP that is followed entails 7 steps from Planning to Acceptance check, that are performed sequentially by the 5 agents that were defined previously.

For instance, in the Software Company system, this is the main description of the SOP governing the Product Manager agent's process: "The Product Manager generates a Product Requirement Document (PRD), as detailed in the specified documentation. This document encompasses goals, user stories, competitive analysis, requirement analysis and requirement pool. Additionally, a competitive quadrant chart is produced. Subsequently, these documents and charts are handed over to the architect for system design." And this operating procedure is encoded with more details and precision inside the code defining the Product Manager role, as illustrated in Figure 7. On line 36, we can see that the agent acts based on an 'instruction' attribute, which is given by "PRODUCT_MANAGER_INSTRUCTION". As it can be observed in Figure 8, this variable contains a concatenation of "ROLE_INSTRUCTION" and "EXTRA_INSTRUCTION". These two variables contain the detailed instructions prompt for the Product Manager agent; the first one is a general instruction inherited from the *RoleZero* base class that defines the inner ReAct working for all agents, and the second one is specific to each designed agent. However, it is important to note that this instruction attribute is in fact a role-level prompt meant to 'shape' the agent's persona. It is a frame for the agent's general behavior and operating mode. The other specific execution instructions are implemented separately in prompts at the action level, inside

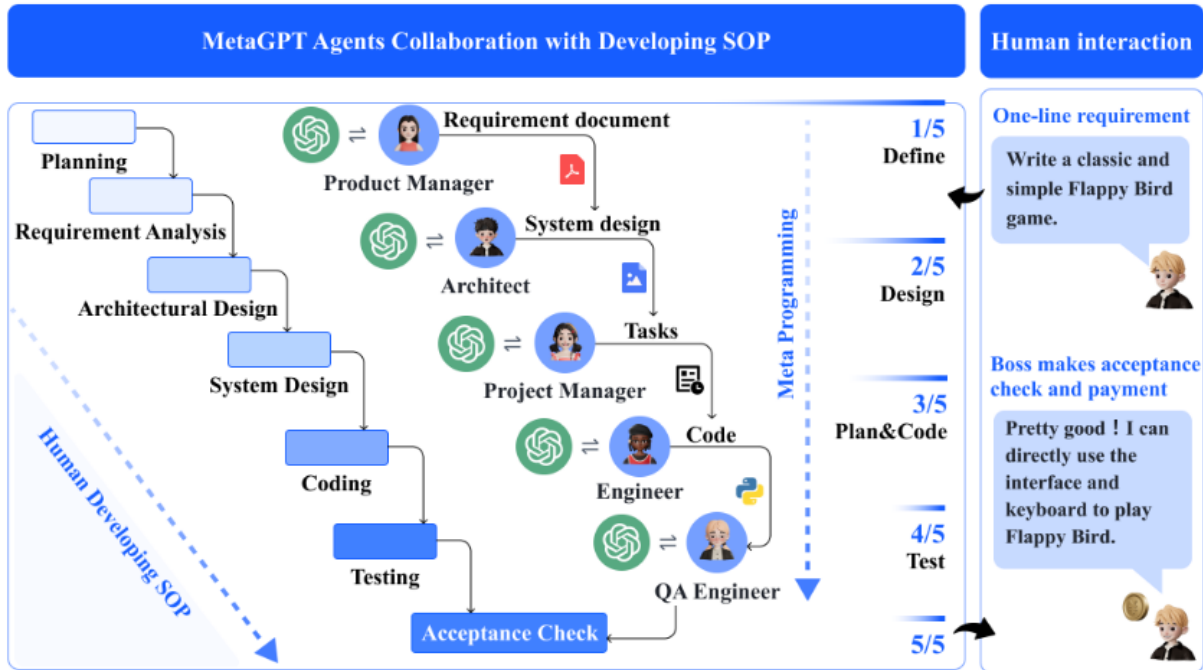


Figure 6 The real-world human software development SOPs used to structure the MetaGPT workflow (reprinted from Hong et al. (2024)).

"PROMPT_TEMPLATE" attributes within each Action class. And then ultimately, when an agent is working, these two layers of prompts are sent with the LLM call, but they serve different purposes: the role-level instruction defines how the agent generally behaves, while the prompt on the action level defines what it concretely does when executing a given task.

While Actions define the procedural steps of the SOP, the *MetaGPT* framework uses Tools to extend an agent's functional reach. While both could initially be confused, Tools are different from Actions in the sense that they are external, modular functions that an agent can call during an execution phase. While an Action represents a more reasoning-related task (e.g., 'Write Code' or 'Analyze Requirements'), a Tool is what provides the agent the technical capability to interact with the outside world. For instance, as can be seen on Figure 7, the Product Manager possesses a tool called 'Browser' which allows him to browse websites and find information on the Internet. These Tools allow the agents to work by not only relying on the LLM's internal knowledge, by grounding their work in real-time data, but above all they allow the agents to interact with and use systems, services and tools that are external to the agentic system.

For more details on the instruction prompts and tools of the Product Manager, as well as the other agents, please refer to the open-source code of the *MetaGPT* system, which is available on the following Github repository: <https://github.com/geekan/MetaGPT>.

```

21 class ProductManager(RoleZero):
22     """
23     Represents a Product Manager role responsible for product development and management.
24
25     Attributes:
26     name (str): Name of the product manager.
27     profile (str): Role profile, default is 'Product Manager'.
28     goal (str): Goal of the product manager.
29     constraints (str): Constraints or limitations for the product manager.
30     """
31
32     name: str = "Alice"
33     profile: str = "Product Manager"
34     goal: str = "Create a Product Requirement Document or market research/competitive product research."
35     constraints: str = "Utilize the same language as the user requirements for seamless communication"
36     instruction: str = PRODUCT_MANAGER_INSTRUCTION
37     tools: list[str] = ["RoleZero", Browser.__name__, Editor.__name__, SearchEnhancedQA.__name__]
38
39     todo_action: str = any_to_name(writePRD)
40
41
42 def __init__(self, **kwargs) -> None:
43     super().__init__(**kwargs)
44     if self.use_fixed_sop:
45         self.enable_memory = False
46         self.set_actions([PrepareDocuments(send_to=any_to_str(self)), WritePRD])
47         self._watch([UserRequirement, PrepareDocuments])
48         self.rc.react_mode = RoleReactMode.By_ORDER
49
50 def _update_tool_execution(self):
51     wp = WritePRD()
52     self.tool_execution_map.update(toolname(writePRD, ["run"]), wp.run))
53
54
55 async def _think(self) -> bool:
56     """Decide what to do"""
57     if not self.use_fixed_sop:
58         return await super()._think()
59
60 if GitRepository.is_git_dir(self.config.project_path) and not self.config.git_reinit:
61     self._set_state(1)
62 else:
63     self._set_state(0)
64     self.config.git_reinit = False
65     self.todo_action = any_to_name(writePRD)
66     return bool(self.rc.todo)

```

Figure 7 The role definition of the Product Manager agent in the original *MetaGPT* Software Company system, retrievable in the `product_manager.py` file stored into the "roles" folder of the open-source Github repository (FoundationAgents, 2024).

4.1.3 Inter-agent communication & memory mechanisms

Unlike other LLM-based multi-agent frameworks that use unconstrained, unstructured natural language to let agents communicate, *MetaGPT* relies on a structured communication approach to avoid the distortion of original information after multiple rounds of communication, and to ensure that each agent works based on the correct information and knowledge it needs. Each agent generates specific outputs based on its SOP instructions and delivers them to the next agent in the workflow through structured outputs like documents and diagrams.

Instead of using one-to-one recurring communication of outputs between agents which is common in other frameworks but inefficient, the *MetaGPT* framework uses a Publish-Subscribe mechanism. Generated information is stored into a global "message pool" shared with all agents. Each agent can publish their structured messages and outputs in this pool, and can also directly access information published by other agents that it needs for its own tasks. To make sure distraction by irrelevant information is avoided, an agent only "subscribes" to the specific information it needs for its tasks, based on role-specific interests defined in their role profiles and instructions. Figure 9 illustrates this communication mechanism. It shows how the Engineer agent uses coding task specifications from the Project Manager, as well as the PRD generated by the Product Manager, which are both published in the message pool, to perform his coding tasks.

In *MetaGPT*, each agent (or Role) also maintains its own local memory. This memory is represented by a list of "Message" objects observed by the agent, that are stored in a "Memory" class object for future retrieval. These memories can be called upon when needed, for example to provide context in an LLM call, or to guide next actions. This retrieval mechanism is based on recency, with a 'k' parameter in the `get_memories()` function that sets the number of most recent memories to return, and that returns all memories by default if k is set to 0. This memory mechanism corresponds to "Short-term memory", but more advanced memory retrieval techniques could also be used and are supported by the framework. More specifically, as mentioned previously, the `RoleZero` class includes an optional `RoleZeroLongTermMemory` module that enables the use of persistent, vector-based memory retrieval across tasks. How-

```

1  from metagpt.prompts.di.role_zero import ROLE_INSTRUCTION
2
3  EXTRA_INSTRUCTION = """
4  You are a product manager AI assistant specializing in product requirement documentation and market research analysis.
5  Your work focuses on the analysis of problems and data.
6  You should always output a document.
7
8  ## Core Tools
9  1. Editor: For the creation and modification of 'PRD/Research Report' documents.
10 2. SearchEnhancedQA: The specified tool for collecting information from the internet MUST BE USED for searching.
113. Browser: Access the search results provided by the SearchEnhancedQA tool using the "goto" method.
12
13## Mode 1: PRD Creation
14Triggered by software/product requests or feature enhancements, ending with the output of a complete PRD.
15
16### Required Fields
171. Language & Project Info
18- Language: Match user's language
19- Programming language: If not specified in the requirements, use Vite, React, MUI, Tailwind CSS.
20- Project Name: Use snake_case format
21- Restate the original requirement
22
232. Product Definition(**IMPORTANT**)
24- Product Goals: 3 clear, orthogonal goals
25- User Stories: 3-5 scenarios in "As a [role], I want [feature] so that [benefit]" format
26- Competitive Analysis: 5-7 products with pros/cons
27- Competitive Quadrant Chart(Required): Using Mermaid
28
293. Technical Specifications
30- Requirements Analysis: Comprehensive overview of technical needs
31- Requirements Pool: List with P0/P1/P2 priorities
32- UI Design Draft: Basic layout and functionality
33- Open Questions: Unclear aspects needing clarification
34
35### Mermaid Diagram Rules
361. Use mermaid quadrantChart syntax. Distribute scores evenly between 0 and 1
372. Example:
38```mermaid
39quadrantChart
40    title "Reach and engagement of campaigns"
41    x-axis "Low Reach" --> "High Reach"
42    y-axis "Low Engagement" --> "High Engagement"
43    quadrant-1 "We should expand"
44    quadrant-2 "Need to promote"
45    quadrant-3 "Re-evaluate"
46    quadrant-4 "May be improved"
47    "Campaign A": [0.3, 0.6]
48    "Campaign B": [0.45, 0.23]
49    "Campaign C": [0.57, 0.69]
50    "Campaign D": [0.78, 0.34]
51    "Campaign E": [0.40, 0.34]
52    "Campaign F": [0.35, 0.78]
53    "Our Target Product": [0.5, 0.6]
54    ...
55
56### PRD Document Guidelines
57- Use clear requirement language (Must/Should/May)
58- Include measurable criteria
59- Prioritize clearly (P0: Must-have, P1: Should-have, P2: Nice-to-have)
60- Support with diagrams and charts
61- Focus on user value and business goals
62
63## Mode 2: Market Research
64Triggered by market analysis or competitor research requests, ending with the output of a complete report document.
65
66### **IMPORTANT** Information Collection Requirements
67
68Must follow this strict information gathering process:
691. Keyword Generation Rules:
70- Infer 3 distinct keyword groups on user needs(Infer directly instead of using tools).
71- Each group must be a space-separated phrase containing:
72    * Target industry/product name (REQUIRED)
73    * Specific aspect or metric
74    * Time frame or geographic scope when relevant
75
76Example format:
77- Group 1: "electric vehicles market size forecast 2024"
78- Group 2: "electric vehicles manufacturing costs analysis"
79- Group 3: "electric vehicles consumer preferences survey"
80
812. Search Process:
82- For each keyword:
83    * Use SearchEnhancedQA TOOL (SearchEnhancedQA.run) collect top 3 search results
84    * Remove duplicate URLs
85
863. Information Analysis:
87- Must read and analyze EACH unique source individually
88- Synthesize information across all sources
89
90- Cross-reference and verify key data points
91- Identify critical insights and trends
92
934. Quality Control:
94- Verify data consistency across sources
95- Fill information gaps with targeted additional research
96- Ensure balanced perspective from multiple sources
97
98### Report Structure
991. Summary: Key findings and recommendations
1002. Industry Overview: Market size, trends, and structure
1013. Market Analysis: Segments, growth drivers, and challenges
1024. Competitor Landscape: Key players and positioning
1035. Target Audience Analysis: User segments and needs
1046. Pricing Analysis: Market rates and strategies
1057. Key Findings: Major insights and opportunities
1068. Strategic Recommendations: Action items
1079. Appendices: Supporting data
108
109### Final Report Requirements
1101. Report must be entirely focused on insights and analysis:
111- No mention of research methodology
112- No source tracking or process documentation
113- Present only validated findings and conclusions
114
1152. Professional Format:
116- Clear section hierarchy
117- Rich subsection content
118- Evidence-based analysis
119- Data visualization where appropriate
120
1213. Content Depth Requirements:
122Executive Summary (500+ words):
123- Key Market Metrics
124- Critical Findings
125- Strategic Recommendations
126
127Industry Overview (800+ words):
128- Market Size and Growth
129- Industry Value Chain
130- Regulatory Environment
131- Technology Trends
132
1334. Quality Standards:
134- Every main section must have 3+ detailed subsections
135- Each subsection requires 200-300 words minimum
136- Include specific examples and data points
137- Support all major claims with market evidence
138
139### Research Guidelines
140- Base all analysis on collected data
141- Include quantitative and qualitative insights
142- Support claims with evidence
143- Maintain professional formatting
144- Use visuals to support key points
145
146## Document Standards
1471. Format
148- Clear heading hierarchy
149- Consistent markdown formatting
150- Numbered sections
151- Professional graphics
152- Output charts using Mermaid syntax
153
1542. Content
155- Objective analysis
156- Actionable insights
157- Clear recommendations
158- Supporting evidence
159
1603. Quality Checks
161- Verify data accuracy
162- Cross-reference sources
163- Ensure completeness
164- Review clarity
165
166Remember:
167- Always start with thorough requirements analysis
168- Use appropriate tools for each task
169- Keep recommendations actionable
170- Consider all stakeholder perspectives
171- Maintain professional standards throughout
172
173---
174
175PRODUCT_MANAGER_INSTRUCTION = ROLE_INSTRUCTION + EXTRA_INSTRUCTION.strip()

```

Figure 8 Instruction prompt for the Product Manager agent in the original MetaGPT Software Company system, retrievable in the product_manager.py file stored into the "prompts" folder of the open-source Github repository (FoundationAgents, 2024).

ever, the Software Company case is mainly based on the short-term memory mechanism that was just described, and these more advanced techniques are unfortunately not widely documented yet in the currently provided documentation of the *MetaGPT* framework.

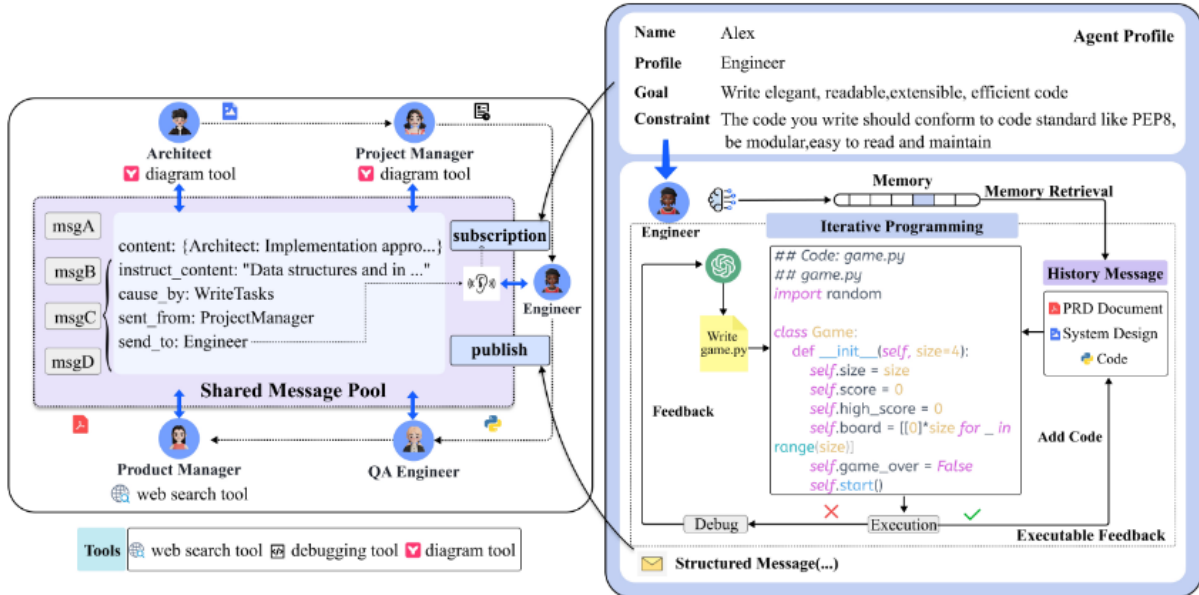


Figure 9 An example of the communication protocol (left) and iterative programming with executable feedback (right) (reprinted from Hong et al. (2024)).

4.1.4 Iterative feedback and self-correction mechanism

In programming tasks, processes of optimization and debugging are common. To enable this, the *MetaGPT* framework incorporates an executable feedback mechanism that will run iterations to optimise the generated code. This is illustrated in Figure 9. As it is depicted, the Engineer agent first generates code based on received PRD and design information. Then it tries to execute this code with corresponding test cases. At each execution, executable feedback is stored into the agent's memory. Based on obtained execution results, it can then continue with the next coding tasks if the test is passed. If not, the Engineer debugs the code using its own execution history and debugging memory in addition to the original PRD and design information. These iterations go on until the test is successfully passed or when a maximum of 3 retries is reached.

In addition to this executable feedback mechanism, the framework also includes a QA Engineer role. This agent is specifically responsible for reviewing and testing the code generated by the Engineer. He takes care of the final quality and acceptance check of the generated application, and sends it back if expected quality is not met.

In summary, this extraction phase shows that *MetaGPT*'s software engineering agentic architecture relies mainly on the following four pillars: 1) strict role personas with specific goals, actions and/or tools, 2) sequential, strict document-driven SOPs,

3) a publish-subscribe communication mechanism, and 4) executable iterative self-correction loops.

In the next section of this chapter, the described mechanisms will be used as a blueprint to perform the actual exaptation, in an attempt to transpose this agentic architecture onto the specific workflow and constraints of the Business Intelligence lifecycle. The term "transposition" is used across the following chapters to refer to the practical application of this "exaptation".

4.2 Conceptual Transposition (Transformation to target domain)

To transpose the agentic Software Engineering architecture towards an agentic BI architecture, we first need to define the precise BI workflow we aim to automate. This will allow the definition of agents to perform the different steps, as well as a general SOP to structure their collaboration, and individual operating procedures, profiles etc. for each agent.

As established in the literature review (cf. section 2.1.2), the BI workflow is a complex multi-phase process. Unlike most frameworks and systems developed in existing literature, which focus on automating specific tasks or isolated parts, this transposition attempts to design an architecture that covers the entire "back-end" BI solution development workflow. The workflow that is targeted for this agentic architecture's SOP thus goes from the initial Requirements Elicitation phase, to the Dimensional Modeling phase and finally to the data transformation pipeline phase. In previous work, such as *DataLab* (Weng et al., 2025) that was presented in the Related work section (cf. section 2.2), the ultimate visualization, dashboarding and reporting phases are also considered in the agentic workflow. In these projects, this is operationalized by using Python libraries, "BI-as-code" or other similar ways, to provide the wanted visualizations using code. While this integrates well in such systems where the main objective is to produce BI analysis 'on-the-go', it doesn't really allow to produce durable visualization/reporting solutions that can be used like a daily information retrieval application. Additionally, in real enterprise environments, these phases of BI development are mostly done using mainstream proprietary BI platforms like Microsoft PowerBI, Qlik, Tableau, etc., which until recently were more complex to bring in interaction with automated agents. As of today, new protocols like MCP have been brought on the market by big BI actors like those, which allow to interact with their BI platforms directly from within LLMs and AI agents. However, the choice was made to not include those last phases in the scope of this transposed architecture, to keep the focus purely on the "back-end" development of the BI data architecture.

Another scoping decision that is important to mention concerns the temporality of execution of the designed agentic workflow. In the original *MetaGPT* system, multi-session, incremental project workflows are supported, which basically allows human users to come back to existing projects or existing parts of objects and work further from there or request updates to previously created artifacts (e.g. adding a new feature to an existing PRD). This is implemented through dedicated branching logic inside the code scripts that define an agent's action and use a versioned filesystem to keep

artifacts across sessions. However, this adds significant complexity and is therefore deliberately left out of scope in this transposed BI architecture. The designed agentic workflow is thought as a one-shot pipeline in which each session starts from a new user request and produces a complete, BI back-end development from requirements elicitation to a working DWH. While this simplification reduces architectural and implementation complexity for the PoC, in the real-life enterprise domain, BI systems actually evolve continuously over time (e.g. addition of new KPIs, extension of existing tables, data source change, etc.). The transposition of *MetaGPT*'s incremental project mechanisms into the BI domain would therefore be very interesting, but is left as a relevant direction for future iterations of the DSR Design cycle, as further discussed in section 5.2.

Additionally, while the author bases this transposition on widely accepted definitions of a typical BI workflow and architecture as presented in the Literature Review (cf. section 2), important is to acknowledge that this remains a not fully neutral choice. As many previous works showcase, including work of previous KU Leuven master students (Van Weyenbergh, 2024), in the BI world a wide range of different architectures and workflows exist, and the one that is applied by the author in this thesis is just one of them. Another author could have chosen to follow a BI workflow differing in some aspects (e.g. different sequence logic, Data lake instead of DWH, etc.), which would ultimately lead to differences in the designed architecture, or would require a more flexible agentic system, capable of adapting to different BI workflows and architectures on user demand.

The following subsections execute the conceptual transposition from extracted concepts to the target domain by applying the four transformation sub-steps defined in section 3.3.

4.2.1 Role and Output Artifact Mapping

To achieve the automation of these steps, the 5 default software engineering roles used in the original *MetaGPT* framework (Product Manager, Architect, Project Manager, Engineer & QA Engineer) are systematically mapped onto their 'functional equivalents' within the BI development workflow. This results in the following 5 BI-specific roles:

- **BI Requirements Analyst (transposed from *MetaGPT* 'Product Manager')**: Conducts conversational requirements elicitation with the business user as well as source-driven elicitation (based on the source data it has access to) to define KPIs, requirements, etc., outputting a formal Business Requirement Document (BRD).
- **BI Data Modeler (transposed from *MetaGPT* 'Architect')**: Based on the BRD, and the source data, decides on the best data schema (star, snowflake, or starflake) to use and translates it into a conceptual and logical dimensional model.
- **BI Solution Architect (transposed from *MetaGPT* 'Project Manager')**: Bridges the logical model to physical execution by defining the external tools and services to use, the technical project structure and a sequential task list to execute to create the needed DWH.

- **BI Analytics Engineer (transposed from *MetaGPT* 'Engineer')**: Performs the actual actions against external tools (data sources, data transformation tools, DWH hosting systems) to perform ETL (Extract-Transform-Load), build the data pipelines and deliver a ready-to-use DWH.
- **BI QA Engineer (transposed from *MetaGPT* 'QA Engineer')**: Reviews the generated DWH, executes automated data and quality tests, and routes execution errors back to the BI Analytics Engineer for self-correction.

This performed methodological mapping of roles is based on the principle of **functional similarity**. This means that each identified software engineering role is transposed to a BI role that performs a similar operational function within the BI workflow. Because these two different domains share multiple functional similarities and analogies in their workflow, a mapping between both can be established. To give this mapping a more concrete academic base, it can be grounded into the concept of "analogical structure-mapping" as defined by Gentner (1983). According to this established method, mapping objects across different domains must be based on relational similarities rather than just matching the individual objects or basing it on superficial features. In this context, not only do the individual roles share functional similarities, but the overarching workflow also preserves relational similarities, which aligns the mapping with Gentner's analogical structure-mapping theory. For example, the Product Manager's function is eliciting requirements and defining the product's goals, which is functionally similar to the BI Requirements Analyst's role of eliciting business requirements and defining KPIs. But their relational structure is also similar in their respective workflow: just as the Product Manager passes requirements to the Architect for system architecture design, the BI Requirements Analyst passes the Business requirements to the BI Data Modeler to define the BI data model and architecture.

A summary of the operated methodological role mapping from source to target domain is given in Table 1.

Along with this transposition of roles, the specific output artifacts that each agent is generating, must also be mapped and redefined for the specific BI context. In the original *MetaGPT* system, the different agents maintain context and a 'common memory' by passing standardized software documents to each other. To preserve this mechanism of structured communication the same document-driven approach is used in the transposed BI architecture, but with BI-specific documents and artifacts.

The artifacts generated by each agent of the transposed architecture are formally (re)defined below. The artifact contents of the first two roles are explicitly guided by the methodologies established by Vaisman and Zimányi (2014), while the others are based on the artifacts of the original *MetaGPT* Software Company case adapter by personal contribution of the author:

BI Requirements Analyst outputs:

- A comprehensive Business Requirement Document (BRD) including:
 - A clear definition of the target end-users of the BI solution (who they are, how they will use it & what their main expectations for the system are).

Table 1

Mapping of original MetaGPT roles and functions to the BI context

Source Role (<i>MetaGPT</i>)	Target Role (Agentic BI)	Functional Transposition (Source → Target)
Product Manager	BI Requirements Analyst	Software PRD Generation → Business Elicitation & BRD Generation
Architect	BI Data Modeler	System/API Architecture Design → Dimensional Schema & Conceptual and Logical Data Modeling
Project Manager	BI Solution Architect	Software Task Planning → DWH Technical Structure & Sequential Task Breakdown
Engineer	BI Analytics Engineer	Application Programming (e.g. Python/C++/etc.) → Data Pipeline & Transformation Execution (e.g. data source extraction, SQL coding in dbt, pandas, Airbyte, etc.)
QA Engineer	BI QA Engineer	Software Integration Testing → DWH Quality Validation & Pipeline Functionality Testing

- A clear definition of goals and needs (one overarching main goal, a list of detailed identified business needs/goals & a list of detailed identified technical needs/goals).
- A clear list of Queries (which capture functional requirements) of the final BI solution (concrete definition of each query and description of the analysis or information retrieval that end-users intend to perform with the BI solution).
- A clear list of needed KPIs with their name, definition, calculation formula and required granularity level.
- A clear list of any elicited nonfunctional requirements (constraints regarding performance, security, availability, etc.).
- A detailed description of the available data sources, their current structure, and their informational content.
- Any specific user requests concerning specific technologies or tools to be utilized or avoided.
- Any other extra information deemed useful for the development of the BI solution.

BI Data Modeler outputs:

- A Dimensional Data Model Specification for the envisioned DWH of the BI solution, outlining the chosen schema (e.g. Star, Snowflake, Starflake) and listing the *Facts*, *Dimensions*, *Measures* and *Hierarchies* with a detailed description for each.
- A high-level Conceptual Schema of the envisioned DWH, formatted according to the Entity-Relations (ER) model.
- A detailed Logical Schema of the envisioned DWH, formatted according to the Relational model.

BI Solution Architect outputs:

- A DWH technical execution plan in structured JSON format, breaking down and listing the necessary tasks to realize the envisioned DWH, including:
 - Instantiation tasks (e.g. Creating the physical DWH environment, establishing connections/APIs to external source systems and transformation tools, etc.)
 - User interaction tasks : points where human-in-the-loop validation or input (e.g. Providing connection credentials) is required.
 - Data modeling, transformation coding and pipeline tasks
 - Any other needed tasks to create the ready-to-use DWH

BI Analytics Engineer outputs:

- An Execution Report containing:

- Logs/status of all tasks performed against the various external systems according to the task execution plan.
- Any warnings or non-blocking errors encountered
- Instructions to connect to the created DWH

BI QA Engineer outputs:

- A Validation Feedback Reports containing:
 - Structural/technical validation & Requirements traceability validation feedback for the created DWH (collected by testing)
 - Precise errors and tasks that need to be fixed
 - Overall validation decision (ACCEPTED/REJECTED)
 - If DWH accepted: DWH connection details/instructions (relayed from BI Analytics Engineer)

4.2.2 Standard Operating Procedure (SOP) Redesign

Now that the new BI roles and their corresponding output artifacts are established, the overarching workflow governing their collaboration must also be redefined. In the original *MetaGPT* framework, Standard Operating Procedures (SOPs) are not just procedures dictated by a central, main control script. Instead, the framework relies on an event-driven, 'decentralized' way of coordinating the collaboration. Agents continuously observe a shared virtual "Environment" (the global message pool that was described in section 4.1) and trigger their own tasks only when they detect the publication of a prerequisite artifact by an upstream role.

To effectively adapt and transpose this mechanism into the target domain of BI, the sequential workflow through which the agents go, is rewritten to reflect the followed BI development workflow. By doing so the global SOP is redesigned, and forms a strict chain of dependent tasks that define exactly which inputs are required to trigger each agent, and which resulting outputs they must necessarily publish back in the shared pool to allow going further along the workflow.

This redesigned global SOP is illustrated in Figure 10, and operates through the following sequential event-driven steps:

1. Phase 1: Requirements Elicitation

- Acting agent: BI Requirements Analyst
- Input trigger: Initial natural language request from the human user.
- Performed action: Engage discussion with human user, and scan the structure of data sources, to iteratively elicit the complete requirements and needs, and write them down in a BRD.
- Output published: Business Requirement Document (BRD)

2. Phase 2: Architectural Data Modeling

- Acting agent: BI Data Modeler
- Input trigger: Publication of a BRD in the Environment.
- Performed action: Analyze received BRD, decide on dimensional model to use, write down and list Facts, Dimensions, Measures and Hierarchies with a detailed description for each. Draw according conceptual schema and logical schema.
- Output published: Dimensional Data Model Specification, Conceptual Schema and Logical Schema.

3. Phase 3: Technical Execution Planning

- Acting agent: BI Solution Architect
- Input trigger: Publication of Dimensional Data Model Specification, Conceptual Schema and Logical Schema in the Environment.
- Performed action: Define all necessary tasks to realize envisioned DWH and write them down as a structured JSON task list which forms the DWH Technical Execution Plan.
- Output published: DWH Technical Execution Plan.

4. Phase 4: Pipeline and DWH Development

- Acting agent: BI Analytics Engineer
- Input trigger: Publication of the DWH Technical Execution Plan *OR* the Validation Feedback Report in the Environment (when it acts after Agent 5 for execution correction).
- Performed action: Execute each task listed in the execution plan to instantiate the DWH, establish needed connections, ask human user needed input, generate executable data transformation code and execute this code in the needed external tools.
- Output published: Execution Report.

5. Phase 5: Testing and validation

- Acting agent: BI QA Engineer
- Input trigger: Publication Execution Report in the Environment
- Performed action: Run quality tests and test queries against created DWH, compare to requirements in the BRD and to Logical Schema. Write Validation Feedback Report (if DWH REJECTED: containing errors and tasks to fix, if DWH ACCEPTED: containing instructions to connect to created DWH and to retrieve BRD, Schemas and other artifacts created in the process).
- Output published: Validation Feedback Report.

Applying the iterative feedback self-correction mechanism:

As it can be seen in Phase 4 and 5, the SOP incorporates a self-correction mechanism to ensure the robustness of the final outputted BI solution, similarly to what is present in

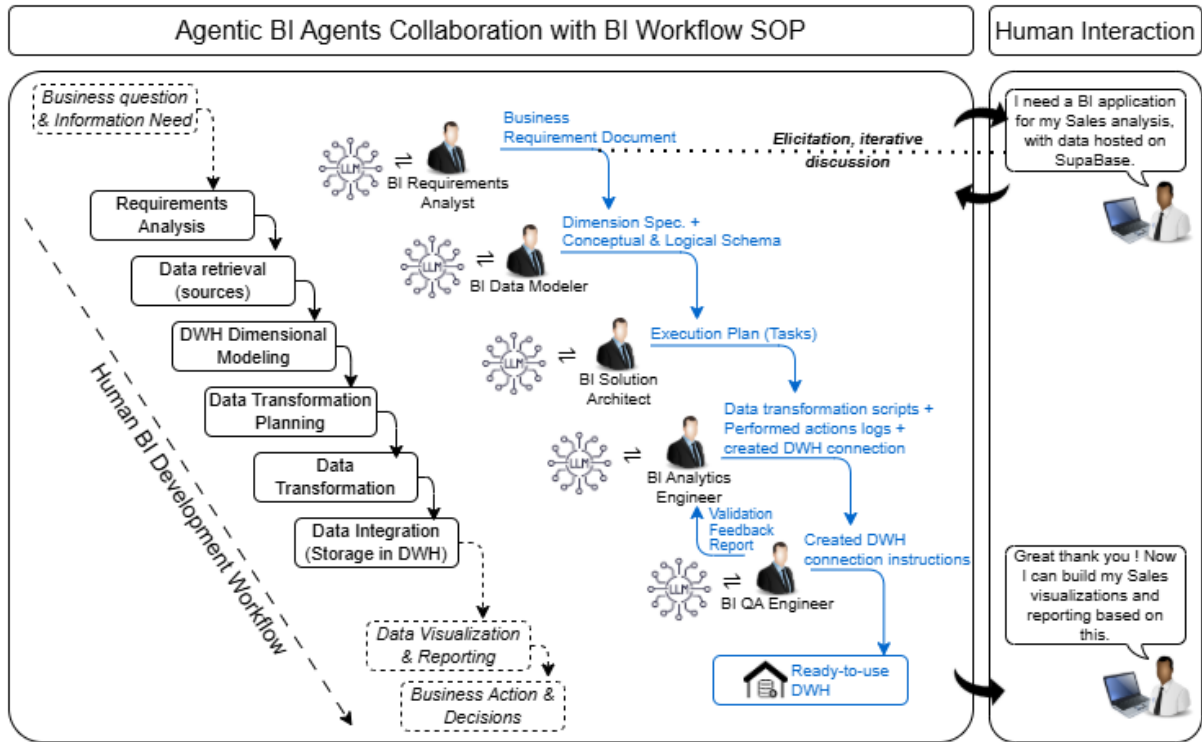


Figure 10 The real-world human BI development SOP used to structure the Agentic BI workflow (adapted from Hong et al. (2024)).

the original *MetaGPT* Software Company system. If the BI QA Engineer detects errors or problems to fix, the DWH validation is rejected, and the execution errors, failed data tests or quality problems are described in the Validation Feedback Report he publishes. The BI Analytics Engineer then observes this artifact in the shared pool, which triggers a revision action, prompting the BI Analytics Engineer to debug and correct the errors, which sequentially triggers the QA Engineer to test the system again, until no errors are detected. When the DWH validation is accepted, a Validation Feedback Report is still published, but containing the instructions to connect to the created DWH and to retrieve the BRD, the schemas and other different artifacts that were produced during the process.

4.2.3 Persona and Prompts Engineering

To make the transition from a high-level, theoretical SOP to an architecture that is really executable, the profile and behaviour of each agent must be strictly formalized. This is because LLMs are inherently probabilistic models meant as general-purpose assistants, and prone to hallucinations or inconsistencies. Therefore, they require rigorously engineered instruction prompts, with operational context and constraints, if we want them to function and act precisely as the enterprise professionals we want (Hong et al., 2024).

In line with step 3 of the Transformation phase defined in section 3.3, this section defines the profile, the specific attributes (name, goal, constraints, tools/actions) and instructions for each agent of the agentic BI system. These prompts definitions will sub-

sequently be used to initialize the agent classes in the transposed codebase. To avoid overflowing this section of the thesis and keep text explanations clear for the reader, the full instruction and action-level prompts are given as an illustration for Agent 1 only. For the following agents, only text design explanations are provided in this section of the thesis, referring to the full prompts for each agent, which are retrievable in Appendix B.

As mentioned in section 3.3, to construct effective prompts, the author relies on those used in the *MetaGPT* original Software Company code, but also on the Prompt Engineering Guide by DAIR.AI (2026). It defines 5 general principles that are followed closely, as well as a number of established prompting techniques, of which some are applied across the agents of this architecture.

First of all, the agent instruction prompts are mostly designed as zero-shot prompts, relying mostly on detailed role instructions and explicit output format specifications instead of real worked out examples. For some specific parts of the designed prompts a little bit of few-shot prompting is used (e.g. to provide a set of example query/analyses that could be part of a BRD). In future iterations of this architecture design, it would be beneficial to try using more few-shot prompting by including more clear examples in the instruction prompts and see if this improves the generated outputs.

In several of the constructed prompts, the Meta-prompting technique is also used. When using this technique the focus is put on the structural and syntactical aspects of the response we expect from the LLM, by providing a clear wanted format or structured template. This is especially used in the prompts of this agentic architecture that define the artifacts agents need to produce and their expected format.

Within each agent, a Chain-of-Thought decomposition technique is also applied: rather than telling them to produce an output artifact directly from just a vague input, each instruction prompt breaks the task into multiple reasoning steps that the agent must go through before producing the final output.

At a higher system-related level, the Prompt Chaining technique is also put in place, by the multi-agent SOP workflow itself. Indeed, in this architecture the structured output artifact of each agent becomes the input context for the next one, thus breaking the "BI back-end development" task into subtasks with each an own agent and prompt. This helps achieve better performance controllability and reliability.

And finally, the iterative self-correction loop that is present between the BI QA Engineer and the BI Analytics Engineer is a direct application of the Reflexion technique. The Analytics Engineer agent observes the result of its action in the Feedback Report received from the QA Engineer, reflects on the cause of failure and then uses that to improve its next attempt.

The following subsections apply these principles consistently across all five agents.

4.2.3.1 Agent 1: BI Requirements Analyst

In the original *MetaGPT* framework, the Product Manager role is responsible for analyzing the user's initial (often vague) prompt and generating a Product Requirement Document (PRD).

In the BI development domain, accepting a one-shot prompt to build a BI architecture would most likely lead to failure, as business users rarely formulate their exact

analytical needs perfectly on the first try and in one sentence. Instead, we really need an agent that can iteratively elicit the complete requirements and needs of the user, by engaging conversation and by analyzing the source data that it has access to. However, this requires a well-thought adaptation, as in the original *MetaGPT* Software Company system the ProductManager receives a single, one-shot UserRequirement message and immediately writes the PRD from it. In fact, this is because the default used Environment loop in *MetaGPT* is designed for sequential, non-interactive pipeline execution, where agents act as a reaction to messages from other agents placed in the Environment shared message pool, and not from humans (except for the initial request message).

Therefore, to enable conversational elicitation within this architecture, the BI Requirements Analyst agent is built using the previously defined `RoleZero` class, in order to use its built-in `reply_to_human` command. This will allow the agent to put his ReAct loop on 'pause' at any point, to send a question or specific clarification request back to the human user, and then resume execution once an answer is received. And all of this, while simultaneously retaining the conversation history in his memory accross conversation turns. By doing so, the agent will iteratively ask structured elicitation questions (covering all elements of the BRD) until he determines that sufficient information has been gathered, and at that precise point he writes the final BRD and publishes it in the shared message pool, which will trigger the next agent in the workflow.

It is worth noting that *MetaGPT* also provides a second, distinct human engagement mechanism : the `is_human=True` parameter. When turned on, it replaces an entire role with a human operator that is then entitled to his responsibilities in the workflow. While this could be used in a future iteration to insert a human validation gate between BRD publication and the triggering of the BI Data Modeler, allowing a business stakeholder to formally approve requirements before technical development begins, it would require to add an extra "BRD Validator" intermediate human role and is thus left outside of the scope of this thesis for simplicity. In specific cases, this parameter could also be used to replace some of the encoded agents by human agents (e.g. in a case where an enterprise would automate everything except the Requirements elicitation part).

Next to conversational requirements elicitation, the BI Requirements Analyst is also designed to autonomously inspect available data sources. To do so, a dedicated "DataSourceInspector" Tool is implemented to support connections to multiple types of databases and flat data files (e.g. a CSV file). Using this, the agent can extract the structure (table names, column names, data types and row counts) of each available source. Connection details for the source (e.g. API credentials) are asked to the human user through conversation and the retrieved schema information is then stored in the agent's memory, which will allow him to ask more informed elicitation follow-up questions based on this before producing the final BRD. These two elicitation capabilities are aligned with both the analysis-driven and source driven requirements specification approaches described by Vaisman and Zimányi (2014) as discussed in the Literature Review (cf. section 2). This brings a big functionality extension compared to the original one-shot requirements *MetaGPT* pattern.

Role attributes:

As mentioned above, the BI Requirements Analyst is built based on the `RoleZero` class. The needed persona attributes for this agent, are defined as visible in Table 2. For simplicity, the agent names used in *MetaGPT*'s original Software Company system have been taken over as-is for all agents in this transposed architecture.

When defining a role class in the *MetaGPT* framework, its main goal and constraints must be given, but also the Tools it can use to perform its tasks, the main Action it needs to execute and the previous agent's Action to observe that will produce the artifact triggering its own start of work (in this case, this is triggered by an initial `UserRequirement` sent by the Human user, as this is the first agent in the workflow). As can be seen in Table 2 `RoleZero` is also included in the list of Tools, in line with what is done in Figure 7 for the Product Manager. This is done to register the foundational capabilities of that base class as accessible functions for the BI Requirements Analyst role. So, this makes `RoleZero`'s internal functions, like the `reply_to_human` command for instance, "discoverable" to the LLM during the agent's `_think()` process of its ReAct loop.

Table 2

Role attributes for the BI Requirements Analyst Agent

Attribute	Value
name	Alice
profile	BI Requirements Analyst
goal	Conduct a structured, iterative requirements elicitation dialogue with the business user, analyze the structure of available data sources and produce a complete, formal Business Requirement Document (BRD) that captures all information needed for the development of a Business Intelligence solution.
constraints	Always use the same language as the business user, during conversation and in the BRD. Never make assumptions about unstated requirements and metrics, but ask instead. Do not continue to BRD writing before all mandatory elicitation topics have been covered. The BRD must strictly follow the defined output format.
tools	RoleZero, Editor, DataSourceInspector
_watch	UserRequirement
todo_action	WriteBRD

Role-level instruction prompt (instruction attribute):

The prompt assigned as value to the `instruction` attribute of the BI Requirements Analyst frames the agent's general behaviour and personal SOP, its elicitation process and way of operating across all its tasks. This value is what is sent in as the general system prompt in every LLM call this agent makes.

It is built following the structure of the original *MetaGPT* framework, thus by concatenating a shared "ROLE_INSTRUCTION" (inherited from `RoleZero` to define the

general ReAct loop behavior, tool call syntax and commands like `reply_to_human`), with a role-specific "EXTRA_INSTRUCTION" which is the instruction prompt that follows below and that is also retrievable in Appendix B.1.

In line with the prompt engineering tips defined by DAIR.AI (2026), the following design principles are applied:

- The **task is broken down** (Chain-of-Thought technique) into 2 clearly separated phases: Requirements elicitation & BRD writing
- Each phase **starts with a direct instruction**, specifying exactly what the agent must do, and clear **text separators** are used.
- All mandatory, needed elicitation topics and BRD sections are listed explicitly to **ensure specificity** and avoid scope assumptions by the LLM
- To **avoid impreciseness**, instead of vague instructions like "gather requirements", concrete delimited ones are used that the LLM can assess (e.g. "proceed to Phase 2 only when all mandatory topics have been covered")
- The prompt specifies **what the agent must do, rather than what it must not do** to avoid negative framing and bring the focus on the details that produce good responses from the LLM.

These same prompting best practices are continuously applied in the definition of the other agents' prompts.

Note that here, on top of the techniques presented previously, the Few-shot prompting technique is also used in the "Queries and analyses" point of the "Phase 1: Requirements elicitation" instructions. This is done by providing an example of 3 queries/-analyses, to make sure the LLM understands what is expected for this elicitation topic. The example used is borrowed and translated from a previous group project report produced by the author and colleagues for the *Business Intelligence* course at the University of Namur (Catale et al., 2025).

BI Requirements Analyst "EXTRA_INSTRUCTION" prompt

You are a senior Business Intelligence Requirements Analyst. Your role is to act as the first agent in a BI development workflow. Your sole responsibility is to conduct a thorough, structured requirements elicitation with the business user and produce a complete, formal Business Requirement Document (BRD) as your only output artifact.

Core tools

1. Editor: For the creation of the final BRD document.
2. DataSourceInspector: For connecting to and inspecting the schema of available data sources. Use this when the user provides connection details for a database or file.
3. `reply_to_human`: For sending question messages to the business user and waiting for their response, during the elicitation phase.

Operating mode

You start working as soon as a UserRequirement is observed. You operate in two strictly sequential phases. Complete Phase 1 entirely before beginning Phase 2.

Phase 1: Requirements elicitation

Before writing anything, conduct a structured elicitation dialogue with the business user using `reply_to_human`. Cover ALL of the following mandatory topics before moving to Phase 2:

1. End-users

- Who are the final users of the BI solution?
- What are their roles, technical level and main expectations for the system?

2. Business goals and needs

- What is the overarching business goal this BI solution must support?
- What are the detailed business needs and goals it must address?
- What are the detailed technical needs and goals it must address?

3. Queries and analyses

- What specific business questions must the BI solution be able to answer?
- What analysis cases or information retrievals must end-users be able to perform with the BI Solution?

Example: “

(a) Analysis and monitoring of renewable energy production

- i. Monitoring changes in energy production (wind and solar) over different time periods (daily, weekly, monthly, annually)
- ii. Analysis of electricity production trends by equivalent period
- iii. Comparison of production by type of wind turbine (onshore/offshore)

(b) Geographical analysis of the performance of wind and solar farms

- i. Ranking of the 5 locations where wind farms perform best and worst
- ii. Ranking of the 5 postcodes where solar panels perform best and worst
- iii. Mapping of energy production by region for wind turbines and solar panels
- iv. Ranking of the 5 best and worst geographical areas for electricity production
- v. Ranking of the 5 best and worst wind farms for onshore and offshore farms

(c) Reliability and accuracy of production forecasts

- i. Analysis of discrepancies between predicted and actual production at different time scales
- ii. Ranking of the 5 geographical areas where forecasts are most reliable and those where they are least accurate

”

4. KPIs and metrics

- Which Key Performance Indicators (KPIs) must be tracked and reported?
- How is each KPI defined and calculated?
- At what level of granularity (e.g. per day, per region, per product) must each KPI be available?

5. Data sources

- What data sources are available (databases, flat files, APIs, etc.)?
- For each source: ask the user for connection details (host URL, API credentials, or the local file path).
- Once connection details are provided, use `DataSourceInspector` to connect to the source and retrieve its data structure (table names, column names, data types, row counts).

- Use the retrieved structure to ask informed follow-up questions (e.g. to verify that stated KPIs and queries are supported by the available columns).
6. Non-functional requirements
 - Are there constraints regarding performance, security, data refresh frequency, or availability?
 - Are there specific technologies, tools or platforms that must be used or avoided?
 7. Additional context
 - Is there any other information relevant to the development of this BI solution?

Rules for Phase 1

- Conduct the elicitation as a natural, professional conversation. Do NOT present all questions at once as a rigid numbered list.
- Group related questions logically and ask them across multiple turns.
- Ask focused follow-up questions when answers are incomplete or ambiguous.
- Base all follow-up questions on what the user has already said, and acknowledge their previous answers before asking the next question.
- NEVER assume or infer values for unstated requirements. Always ask instead.
- Proceed to Phase 2 only when all seven mandatory topics above have been sufficiently covered so that the BRD can be written completely.

Phase 2: BRD generation

Once elicitation is complete, inform the user that you now have sufficient information and will produce the BRD. Use the Editor tool to write and save the BRD as a structured markdown document. Do not ask further questions in this phase.

The BRD must strictly follow the following structure:

Business Requirement Document (BRD)

1. Project overview

- Project name, date

2. End-user definition

- Who they are, their roles, technical level, and main expectations

3. Business goals and needs

- One overarching main goal
- Detailed list of business needs/goals
- Detailed list of technical needs/goals

4. Queries and analyses

- Numbered list of all identified queries, each with: description | analysis or information retrieval it concerns

5. KPIs and metrics

- List of all KPIs with: name | definition | calculation formula | required granularity

6. Data sources

- For each source: name | type | schema summary (from DataSourceInspector output) | content description | data quality notes | update frequency

7. Non-functional requirements

- Performance, security, availability, technology constraints

8. Additional information and open questions

- Any other relevant information
- Any remaining ambiguities or open questions flagged explicitly for human review

Quality standards

- Every section must be fully completed. Do not leave sections empty or with placeholder text.
- Base all content exclusively on information provided by the user during the elicitation conversation and on schema data retrieved by DataSourceInspector. Do not invent or infer any information that was not explicitly stated or observed.
- Ensure all queries listed in Section 4 are linked to at least one goal in Section 3 or one KPI in Section 5.
- Use clear, unambiguous, professional language throughout. Use clear requirement language (Must/Should/May) for Section 3 and 7.
- Use the same language as the business user.

Action-level Prompt template:

As mentioned in Section 4.1.2, next to the role-level instruction prompt defining the agent's general behaviour, in *MetaGPT*, other execution instructions that are specific to subtasks of each agent are implemented separately in prompts at the Action level. These action-specific instructions are implemented separately inside "PROMPT_TEMPLATE" attributes in each Action class.

For the BI Requirements Analyst agent, this only concerns the `WriteBRD` Action class which enables the agent to generate the final BRD artifact once Phase 1 of Elicitation (as it is defined in the Role-level instruction prompt) is completed. To guide its execution, the following prompt is inserted inside the "PROMPT_TEMPLATE" attribute of the `WriteBRD` Action class to provide the LLM with the full elicitation conversation (with the human user) history, summaries of the source data structure that were extracted and specific formatting instructions for the BRD. This Action prompt is given below.

Important to mention is that, this transposed `WriteBRD` Action class in the agentic BI architecture is quite different than the `WritePRD` action of the original *MetaGPT* Software Company system. In the original system of the source domain, the `WritePRD` handles multiple types of scenarios (new requirements, updates or bugfixes), uses structured `ActionNode` objects, reads/writes in a filesystem named `ProjectRepo`, generates Mermaid diagrams and a PRD in JSON format. This is rather complex and the transposed `WriteBRD` action follows a simpler working. Since the elicitation history is already available in the agent's memory and the BRD is just a single markdown document with no special branching logic, a straightforward `PROMPT_TEMPLATE` prompt sent to the LLM should be sufficient. In addition, the multiple scenario handling of the original *MetaGPT* system integrates the possibility to update already produced requirements or change them to correct bugs, but this is out of the scope of this thesis in which requirements are considered 'finished' and final after the elicitation phase.

WriteBRD "PROMPT_TEMPLATE" prompt

Task

You are completing Phase 2 of your instructions. The elicitation dialogue with the business user is now complete and all mandatory topics have been covered. Based on the information collected below, write a complete and formal Business Requirement Document (BRD).

Elicitation conversation history

{elicitation_history}

Data source schemas retrieved during elicitation

{schema_summaries}

Instructions

- Write the BRD strictly following the output format defined in your role instruction.
- Base all content exclusively on information provided by the user in the elicitation history above and on data source schemas retrieved by DataSourceInspector as shown above.
- When the user's answers are ambiguous or incomplete despite follow-up, flag this explicitly in Section 8 as an open question requiring future clarification.
- Ensure all queries in Section 4 are linkable to at least one goal in Section 3 or KPI in Section 5.
- Always use the same language as the business user, and only that language.
- Output the complete BRD now, starting directly with the document title, without any explanation or other content before the document.

4.2.3.2 Agent 2: BI Data modeler

In the original *MetaGPT* Software Company System, the Architect agent makes the bridge between requirements and actual software engineering, by translating the received PRD into a software system design and architecture (data structures, APIs, file structures, program calls, etc.).

In this transposed BI architecture, the BI Data Modeler takes on the functionally 'equivalent' role. He takes the BRD, that was published in the shared message pool by the BI Requirements Analyst, as input and transforms this into a dimensional architecture design for the future Data Warehouse that will serve the BI solution. More concretely, this includes choosing the most appropriate dimensional schema type (star, snowflake or starflake), identifying what are the facts, dimensions, measures and hierarchies that are needed to answer the requirements of the BRD and also generating both a conceptual and a logical data schema for the envisioned DWH.

Some design decisions that were iteratively made to transpose this role from the original Architect agent are worth noting, before continuing to the agent's attributes and prompts definitions.

First of all, concerning the way this agent is guided for his dimensional schema type decision, multiple different approaches have been considered. A first option was to base the agent's decision-making in a RAG module, in which he could retrieve relevant explanations and examples from dimensional modeling literature (e.g. (Kimball & Ross, 2013), (Vaisman & Zimányi, 2014)) inputted into it. The *MetaGPT* framework does actually support such an integration natively through its RAG module where a *SimpleEngine* class retrieves info in a vector store created from documents, as illus-

trated in the *MetaGPT* documentation examples (FoundationAgents, 2024). However, two practical considerations led to not using this approach in the current architecture. Firstly, as these dimensional modeling sources are copyrighted, published books, ingesting them into a vector store for use in enterprise-related contexts forms a form of redistribution that go along with significant risks if the RAG system reproduces copyrighted content freely. Secondly, the schema-type decision is in essence a bounded problem with only a small number of criteria to take into account. Using RAG retrieval only to get to these bits of information would arguably add infrastructural complexity into this agentic architecture design without delivering really proportional benefits. Therefore, the approach that was finally chosen in this thesis research is to encode specific decision criteria directly into the agent's instruction prompt, derived from the established dimensional modeling literature by Kimball and Ross (2013) and Vaisman and Zimányi (2014). The use of RAG over a specialized knowledge base is however still left as a possible improvement in future iterations of the Design cycle of the followed DSR process, as mentioned in section 5.2.

A second key design choice concerns the modeling notation used by the agent to produce the conceptual and logical schemas. Vaisman and Zimányi (2014) have explored this subject extensively and even propose their own *MultiDim* conceptual model notation which is specifically tailored to DWH multidimensional design. However, they also acknowledge in their book that the 'classical' Entity-Relationship (ER) model remains the most widely used conceptual model for database design and is broadly seen as a standard across both academia and industry. And for the logical level, they adopt the Relational model which is also seen as the dominant standard for logical design of databases. As Vaisman and Zimányi's own *MultiDim* notation requires specific graphical elements and symbols that are often not inherently provided by today's code-based diagram drawing languages and given that current LLMs are better at generating standard ER and Relational models due to their wide presence in their training data, the ER model is used as the conceptual model and the Relational model for the logical schema in this transposed architecture. To produce both these schemas, the system relies on *Mermaid*, and more precisely 'erDiagram' code blocks (Sveidqvist et al., n.d.). This mirrors the pattern of the original *MetaGPT* Architect role, who first produces a markdown system design followed by *Mermaid* 'classDiagram' and 'sequenceDiagram' visualizations. Other code-based diagram drawing languages or tools such as *PlantUML*, or even *DBML/DBDiagram* (which are specifically addressing database modelling) could have also been used for this (Holistics Software, 2026; Roques, 2026). However, since *Mermaid* was already used and implemented in *MetaGPT*'s source code and possesses the 'erDiagram' code blocks that should be able to produce the needed schemas, this implementation is used for simplicity. If the produced schemas appear to be unfit or not sufficient, *Mermaid* could be replaced by one of the aforementioned tools in future design iterations.

A third important decision to notice is about also giving the BI Data Modeler agent direct access to the available data sources, in addition to the access that is already provided to the BI Requirements Analyst. This would allow the BI Data Modeler to verify directly that dimension and fact tables he proposes can actually be constructed from the source data and could help him decide on data types, key candidates, etc.

However, the choice was finally made here to not give this access. This is because it would break the document-driven SOP principle that was presented in section 4.1.2 as one of the core elements of the *MetaGPT* architecture. Therefore, instead of a direct data source access, the BI Data Modeler relies exclusively on the BRD that was published by the BI Requirements Analyst, which must include the data source schema summaries in Section 6 of that BRD document. This preserves the structured publish-subscribe communication workflow defined in the original framework, prevents different interpretation of source data structure (only one single source of truth) and keeps each agent focused on its own core responsibility and subtasks.

And finally, about the task execution structure, the BI Data Modeler agent is designed to work with one single Action class, which groups the production of all three artifacts (the dimensional model specification, the conceptual ER schema and the logical Relational schema) in one execution. This, again, perfectly mirrors the inner working of the original *MetaGPT* WriteDesign Action class that also generates a system design in markdown format, along a class and a sequence diagram, all in one single coordinated execution. Thereby, using this consolidated structure instead of splitting the execution across three separate Action classes stays consistent with the DSR exaptation methodology followed in this thesis, which aims to keep the adequate system patterns of the source framework while adapting them to the target domain.

Role attributes:

The BI Data Modeler agent is also built based on the `RoleZero` class, just like the BI Requirements Analyst and like the original *MetaGPT* Architect role from which it is transposed. The needed persona attributes for this agent, are defined as in Table 3.

The only tool that this agent will need is the Editor tool; to produce the Dimensional Data Model Specification. The drawing of the two schemas is handled in the Write-BRD Action using *Mermaid* text formatting (instructed to the LLM in the action-level prompt) that can be rendered to visual files using external drivers such as "mermaid-cli" or "Playwright/Pyppeteer" that are installed together with the *MetaGPT* system. The `RoleZero` tool is also added again, to ensure the agent has access to all functionalities it would need from the base `RoleZero` class.

Table 3*Role attributes for the BI Data Modeler Agent*

Attribute	Value
name	Bob
profile	BI Data Modeler
goal	Translate the Business Requirement Document (BRD) produced by the BI Requirements Analyst into a complete dimensional design for the future Data Warehouse. This includes the choice of an appropriate dimensional schema type, the identification of all required facts, dimensions, measures and hierarchies and the production of a conceptual ER schema and a logical Relational schema for the envisioned DWH.
constraints	Always use the same language as the original BRD. Base all design decisions exclusively on the content of the published BRD. Never assume or invent requirements that are not stated. The chosen dimensional schema type must be explicitly justified using the decision criteria provided in the role instruction. The produced conceptual and logical schemas must strictly follow the Mermaid erDiagram syntax rules defined in the role instruction.
tools	RoleZero, Editor
_watch	WriteBRD
todo_action	WriteDataModel

Role-level instruction prompt (instruction attribute):

As in Agent 1 and in the original *MetaGPT* framework, here too the role-level instruction prompt is constructed by putting together the inherited "ROLE_INSTRUCTION" from `metagpt.prompts.di.role_zero` with a role-specific "EXTRA_INSTRUCTION" that defines the BI Data Modeler agent's persona, operating mode, design rules and specific output format.

The structure of the "EXTRA_INSTRUCTION" prompt, that can be retrieved in Appendix B.2, is organized into three separate sets of explicit design rules, which takes over the layered rule structure that is also used in the original *MetaGPT* instruction prompt for the Architect agent:

1. Decision criteria for choosing between star, snowflake and starflake dimensional schema types.
2. General dimensional modeling rules that guide the agent for the definition of facts, dimensions, measures and hierarchies.
3. *Mermaid* 'erDiagram' syntax requirements to guide in drawing the conceptual and logical schemas.

To write these different prompt parts, multiple sources were used. For layers 1 and 2 the rules were derived from the author's own knowledge acquired in BI and database courses at the University of Namur (Linden, 2025), as well as established dimensional modelling literature that was mentioned previously (Kimball & Ross, 2013; Vaisman & Zimányi, 2014), to make sure the agent's design decisions are grounded in academically established practices. For layer 3, Sveidqvist et al. (n.d.) is used as source to write out syntax rules.

The same DAIR.AI (2026) prompting principles that were discussed in section 4.2.3.1 are also applied here:

- **Chain-of-Thought technique:** the LLM is instructed to follow four explicit sequential reasoning steps (analyze BRD → choose schema type → identify facts/dimensions/measures/hierarchies → produce artifacts).
- Each step **starts with a direct instruction**, specifying and listing exactly what the agent must do, and clear text separators are used.
- All rules for the dimensional schema choice and for the identification of facts/dimensions/measures/hierarchies are listed explicitly to **ensure specificity** and avoid assumptions by the LLM. The expected format for each artifact to produce is also explicitly defined.
- To **avoid impreciseness**, a concrete Operating mode specifying when exactly to go to the next step, and Quality standards for the delivered artifacts are specifically defined.
- When possible, the prompt specifies what the agent must do, rather than what it must not do to avoid negative framing and bring the focus on the details that produce good responses from the LLM.

Note that, while Zero-shot prompting is mostly used, here too a little bit of few-shot prompting is used to provide examples of good *Mermaid* syntax code for drawing the schemas, to the LLM.

Action-level Prompt template:

Once the BRD has been observed in the shared message pool, the BI Data Modeler agent executes the WriteDataModel Action, which contains the action-level prompt retrievable in Appendix B.2 to guide its execution. It gives the LLM the full content of the produced BRD as context, as well as clear instructions needed to produce the three output artifacts (by referring to role-level instructions).

As it was also the case for the WriteBRD action of the BI Requirements Analyst, the WriteDataModel action is intentionally a bit simpler than the original *MetaGPT* WriteDesign action that includes the handling of multiple different execution scenarios. As mentioned as a scope limitation in 4.2, in this architecture we don't use this branching logic. Therefore, the three output artifacts are produced in one single coordinated execution, and a straightforward PROMPT_TEMPLATE sent to the LLM to guide its action is sufficient.

4.2.3.3 Agent 3: BI Solution Architect

In the original *MetaGPT* Software Company System, the Project Manager agent takes the system design produced by the Architect as input and produces a coding execution plan based on this. This takes the form of a structured list of sequential coding tasks in a JSON file that the Engineer will subsequently execute one by one.

In this transposed agentic BI architecture, the BI Solution Architect plays the functionally similar role. He takes the Dimensional Model Specification, the Conceptual Schema and the Logical Schema that BI Data Modeler produced as inputs and then translates these into a structured 'DWH Technical Execution Plan' that defines exactly all the tasks that must be performed to create the envisioned DWH that the human user needs. He also specifies the order in which these tasks must be performed, which is very important to avoid fails (e.g. a data transformation task would not be possible to execute if the connection with the data source is not first established).

As for the previous agents, the performed transposition to this agent's working required several design decisions, of which some should be made clear before continuing to the definition of its attributes and prompts.

A first important observation concerns the BI architecture that is utilized in the workflow supported by the designed system. As presented in the literature review (cf. sections 2.1.3 & 2.1.2), the BI landscape has progressively moved from the traditional Extract-Transform-Load (ETL) pattern (where data is transformed outside the DWH before being loaded into it), towards the more modern approach of Extract-Load-Transform (ELT), where raw data is loaded into the DWH first and then transformations are applied to it inside the DWH. In this transposed agentic architecture, the choice was made to assume produced solutions exclusively use the ELT architecture for two different reasons.

Firstly, ELT is today's dominant approach in modern enterprise BI infrastructures and is used in many transformation tools like *dbt* or *Airbyte*. It is widely considered as a standard for cloud-native data stacks.

Secondly, ELT operates with a clear separation between ingestion and transformation phases: all raw data is first loaded into temporary tables within the DWH by an ingestion tool, and then a transformation tool applies the needed transformations to populate all the final dimensional tables.

This clean separation maps very well onto a multi-task agentic workflow like this one because each task can be then assigned easily to the appropriate tool with explicit dependencies between them. On the other hand, ETL has more blurry boundaries between ingestion and transformation phases with separated transformation followed by ingestion for each table. This requires using transformation tools followed by ingestion tools iteratively for each table and makes task decomposition more complicated. The case where a user would explicitly want to use a traditional ETL pipeline is therefore deliberately left out of scope for this thesis and acknowledged as a possibility for future improvement in section 5.2.

A second important design decision concerns the role-level instruction prompt for this agent. In the original *MetaGPT* framework, the Project Manager role is very simple at the role-instruction level. In the Software Company system, the instruction attribute

for this role contains just a single one-line instruction ("Use WriteTasks tool to write a project task list"), and the actual task-breakdown instructions are almost entirely given inside the action-level prompt of its WritePlan Action class. This is sufficient in the original software engineering context because LLMs often handle software task breakdown quite well based on their training data alone, and because the available task type categories in the original framework are small and well-known.

In the BI domain however, there is significantly more complexity in the planning of the different DWH creation tasks. It requires to discern multiple diverse task categories (e.g. DWH instantiation, source connection setup, raw data ingestion, data transformation, etc.), with explicit dependencies between them. When automated by an agentic system, it also involves the integration of multiple distinct external tools and human-in-the-loop checkpoints to collect connection credentials for some of these tools. This added complexity brings the need for a more extended role-level prompt than in the original framework, to encode the BI-specific task categories, the task dependency logic and the clear rules for tool selection.

Moving forward, concerning the format of the DWH Technical Execution Plan that this agent produces, it is important to highlight that, as briefly mentioned in section 4.2.1, this artifact is produced as a structured JSON document. This choice is a direct transposition of what is also done in the original *MetaGPT* system and is a practical necessity to streamline the action that the next agent (BI Analytics Engineer) will undertake. Indeed, given that this one must be able to automatically iterate through the tasks, understand their dependencies before execution, and then dispatch each of them to the appropriate Tool classes (that will call external tools) based on their specific task details, a simple markdown document would not allow this easily. Therefore, a JSON list of tasks is produced with each task conforming to the following schema:

- `task_id`: unique identifier for the task within the plan
- `dependent_task_ids`: list of task IDs that must be completed prior to this task
- `instruction`: natural-language description of what the role must do in this task
- `task_type`: one value from a closed list of BI-specific task categories (see below)
- `tool`: name of the external tool selected to execute the task (when applicable)
- `tool_args`: dictionary of tool-specific arguments needed for execution (when applicable)

The two last fields are not present in the original *MetaGPT* Project Manager's JSON output, and are an adaptation brought for this transposed architecture. This addition is a direct consequence of the difference between the source and target domains. As in the software engineering context, the Engineer agent operates within a single execution environment (which is typically a Python interpreter and the local filesystem), the task type alone provides enough information to determine how the task should be executed. But, as explained further in text, in the BI domain the Analytics Engineer must also connect to several external tools/systems to execute some of his tasks (e.g. a TRANSFORMATION task could be executed using dbt, or Airbyte or some other tool).

The `tool` field thus specifies the tool that must be used for the task, and the `tool_args` field carries the specific configuration information that is needed to execute the task with that tool (e.g. database connection string, source file path, etc.).

Continuing on this JSON list of tasks, another design adaptation compared to the original *MetaGPT* system is important to bring forward. As mentioned, the same pattern as the original system's WritePlan Action class is followed, in which the LLM can only assign task-types that are within a predefined 'TaskType' enum list to prevent hallucinated other task categories that downstream agents cannot handle. For this transposed architecture, a dedicated new `BITaskType` enum is defined. Six task categories are included in this enum which were derived from the artifact specification in section 4.2.1, which was itself defined based on the BI development workflow followed in this design and on the inner working of the ELT method (IBM, 2021b). Table 4 describes these six task types.

Table 4

The 6 task types included in the `BITaskType` enum, with description

Task type	Description
INSTANTIATION	Creating a new instance of an external tool or environment that the BI workflow will use (e.g. initializing a DuckDB database file, provisioning a Supabase project, opening a new dbt project)
CONNECTION_SETUP	Establishing connections between the DWH or source data and external transformation/ingestion tools (e.g. configuring dbt profiles, setting up Airbyte connectors, configuring a pandas connection)
CREDENTIAL_REQUEST	Specific human-in-the-loop checkpoints where the agent must collect credentials, API tokens or local file paths from the user, necessary for connection, before proceeding
SCHEMA_CREATION	Data Definition Language (DDL) execution tasks to create the empty fact and dimension tables in the DWH, based on the Logical Schema produced by the BI Data Modeler.
DATA_INGESTION	Loading raw data from the source systems into temporary staging tables inside the DWH (Extract & Load steps of the ELT pattern)
TRANSFORMATION	Running SQL-based data transformations inside the DWH and populate the dimensional fact and dimension tables from the staging tables (Transform step of the ELT pattern)

And finally, a fifth important design decision to bring forward concerns where exactly, in the overall agentic workflow, the choice of the external tools (which DWH technology, which transformation tool, etc.) that must be used to construct the DWH is made. This could be done 'early' by the BI Requirements Analyst, or 'late' by the BI

Analytics Engineer himself right before using the external tools/systems, but instead, in this transposed architecture, that responsibility is deliberately assigned to the BI Solution Architect. This is because, coming right after the two prior agents, the BI Solution Architect has access to all the information needed to make tool decisions in context: the BRD (including possible tool/system preferences of the user), the dimensional design, and knowledge of the available tool options (based on Tool class integrations that are implemented). Tool choices are recorded as the `tool` field on each task in the JSON plan that the BI Solution Architect outputs. This makes the BI Solution Architect agent the only tool-selection authority of the workflow, which simplifies the responsibilities of the downstream BI Analytics Engineer to pure task execution.

Role attributes:

Like the previous two agents and the Project Manager role from which it is transposed, the BI Solution Architect is also built on the `RoleZero` base class. The needed persona attributes are defined in Table 5.

Here again, the only tool needed by this agent is the Editor tool; to produce the DWH Technical Execution Plan. The `RoleZero` tool is also added again, to ensure the agent has access to all functionalities it would need from the base `RoleZero` class.

Table 5

Role attributes for the BI Solution Architect Agent

Attribute	Value
<code>name</code>	Eve
<code>profile</code>	BI Solution Architect
<code>goal</code>	Produce a complete, dependency-ordered DWH Technical Execution Plan task list in structured JSON format, selecting the appropriate external tools for each task and flagging all human-in-the-loop credential collection checkpoints.
<code>constraints</code>	Always use the same language as the original BRD. Base all planning decisions exclusively on the BRD and on the dimensional design artifacts published in the shared message pool. The execution plan must be produced as valid JSON conforming to the defined schema. Every task must be assigned exactly one task type from the <code>BITaskType</code> enum. Every task that requires an external tool must have a non-empty tool field. Tasks must be ordered so that all dependencies are resolvable, and no task may depend on a task with a higher task ID.
<code>tools</code>	RoleZero, Editor
<code>_watch</code>	WriteDataModel, WriteBRD
<code>todo_action</code>	WriteExecutionPlan

Role-level instruction prompt (instruction attribute):

As for the previous agents, the role-level instruction is also constructed here by concatenating the inherited "ROLE_INSTRUCTION" with a role-specific "EXTRA_INSTRUCTION" defined in the prompt available in Appendix B.3.

As noted previously, the role-level instruction here is significantly 'richer' than the one-line instruction that is used in the original *MetaGPT* Project Manager. This is to address the higher complexity of task execution planning in the BI domain. The three sets of rules encoded in the instruction prompt are:

1. The tool-selection logic to make the LLM able to choose correct tools for each task
2. The BITaskType enum with descriptions to make the LLM able to assign each task a correct type
3. the JSON task schema specification and dependency-resolution rules to make sure the DWH Technical Execution Plan file is written correctly

The same DAIR.AI (2026) prompting principles are still applied and in particular:

- **Chain-of-Thought technique:** the task is decomposed into three explicit sequential reasoning steps (Select tools → Identify tasks → Order & resolve dependencies).
- Each step **starts with a direct instruction**, specifying and listing exactly what the agent must do, and clear text separators are used.
- **Specificity:** all task types and their descriptions are fully listed, and the JSON format of the output artifact (the DWH Technical Execution Plan) is precisely specified.
- To **avoid impreciseness**, a concrete Operating mode specifying when exactly to go to the next step, and Quality standards for the delivered artifact are specifically defined.
- Constraints are most as possible framed as positive instructions (what the agent must do, rather than what he must not do).

Action-level Prompt template:

To guide Agent 3's execution, an action-level prompt is inserted inside the "PROMPT_TEMPLATE" attribute of the WriteExecutionPlan Action class to provide the LLM with the full context needed to produce the DWH Technical Execution Plan, which is its output artifact.

The WriteExecutionPlan action-level prompt closely follows the three-block structure of the original *MetaGPT* WritePlan action:

1. Input context (explanation that it is completing its task as BI Solution Architect, followed by BRD, Dimensional Model Specification and Logical Schema).
2. Descriptions of all available task types to choose from.

3. Task description, referring to the JSON output format specification defined in the role-level prompt.

It is called once all dimensional design artifacts and the BRD are available in the shared message pool, which brings the BI Solution Architect agent into action. This written action-level prompt is retrievable in Appendix B.3.

4.2.3.4 Agent 4: BI Analytics Engineer

The BI Analytics Engineer agent is the most operationally complex agent in this transposed architecture. This agent is responsible for the 'translation' of the DWH Technical Execution Plan provided by the BI Solution Architect, into *actual* executed tasks. To do so, it interacts with multiple external tools and systems for data warehousing, ingestion, transformation, etc. and outputs an Execution Report after operation is complete.

In the original *MetaGPT* framework, two role implementations exist for the Engineer agent: Engineer & Engineer2. The first one uses the base `Role` class and a fixed, rigid SOP, while Engineer2 is a more modern rewrite of the role that uses `RoleZero` as its base class and operates as a more autonomous ReAct agent. Another big difference is that the Engineer role pre-builds a list of `WriteCode` Actions and then iterates through them mechanically, while the Engineer2 reasons at each step on what to do next, selects tools dynamically from a registered tool set and defines its primary execution method (`write_new_code`) directly in the role class itself as a method 'decorated' with `@register_tool` rather than in a separate Action subclass. In simpler words, this means that the engineer's 'code writing' ability is built right into the class like a built-in command that an LLM can trigger when needed. The `@register_tool(include_functions=["write_new_code()"])` line of code, basically makes the specific `"write_new_code()"` function defined in the same role definition code script callable, without exposing other internal functions in the script to the LLM.

For this architecture transposition, the choice was made to work from the Engineer2's architectural pattern. Indeed, as the BI Analytics Engineer must be able to dispatch heterogeneous task types to different external technologies/systems, to handle exceptions, to pause execution for credential collection from the human user, and also to respond to corrective feedback from the BI QA Engineer, this wouldn't fit the more static model of the simpler `Role` base class. In consequence, the BI Analytics Engineer inherits from `RoleZero`, uses a long instruction prompt that is sent to the LLM at each thinking step of the ReAct loop, and the same `write_new_code()` pattern as in the original *MetaGPT* system is applied through the `execute_BI_task()` method. This function is defined directly in the BI Analytics Engineer role class and 'decorated' with `@register_tool`, making it callable by the LLM within the ReAct loop. Concretely, `execute_BI_task()` acts as a Tool dispatch router: when it is called with a task object from the JSON execution plan, it reads that task's `task_type`, `tool` and `tool_args` fields and dispatches execution to the appropriate registered Tool class (e.g., `DuckDBExecutor` for a `SCHEMA_CREATION` task, `DbtRunner` for a `TRANSFORMATION` task, etc.). For `CREDENTIAL_REQUEST` tasks, it calls `RoleZero.reply_to_human`, waits for the human user's response, and then stores the returned credential or connection instructions in the execution context before returning. After each Tool dispatch, the method returns an execution result (success or failure status, row counts, error messages, test

results) which the LLM observes in the next ReAct step in order to decide what to execute next. A key architectural point to understand is that `execute_BI_task()` does not itself perform database operations or transformations: those are performed by the individual Tool classes it calls. The LLM is responsible for selecting which task to execute next; `execute_BI_task` then routes that task to the correct Tool and returns a result that the LLM can observe before continuing. In section 4.2.4 a deeper explanation is given about the concrete implementation of the Tool classes used by this agent, including how they handle multi-step interactions with external systems that are sometimes needed to execute tasks (e.g. for one single TRANSFORMATION task it might be necessary to connect to a dbt project, then write the needed transformation code, then try to execute that code in dbt and possibly correct it, etc., each needing observation and reasoning between them).

The maximum number of ReAct iterations (`max_react_loop`) is also set high as a high number of tool calls will be required to execute a full ELT process.

While this agent is strongly based on how the original framework's Engineer2 functions, 2 specific adaptations needed for the BI domain of this designed architecture should be brought forward. First, unlike the software engineering domain where all final output artifacts are code scripts written into a local filesystem using only one consistent tool, in the BI development domain, multiple different task types must be executed using multiple different external systems (e.g. a connection setup task to connect to source databases, a transformation task to make data transformations via an external system, etc.). As it was defined in section 4.2.3.3, it is the BI Solution Architect that defines the tool that will be used for each task, and that encodes this in the DWH Technical Execution Plan. But, depending on the task type, the BI Analytics Engineer must follow different specific rules, which are given per task type in the role-level instruction prompt.

And secondly, unlike the original framework's agent, the BI Analytics Engineer must be able to pause execution, in order to ask connection details or credentials to the human user (CREDENTIAL_REQUEST tasks). To do so, it calls `RoleZero.reply_to_human` (which was also used in Agent 1 for conducting an elicitation conversation), it stores the returned credentials or instructions and only then continues execution. This specific interaction pattern has not really an equivalent in *MetaGPT*'s original Engineer2 agent, which operates entirely autonomously. It thus represents a real domain-specific extension whose need is caused by the reality that in a BI ELT workflow, connection is made with multiple external data systems for which access is often restricted by credentials or connection tokens which must therefore be given to the agent by the human user.

A final design decision worth mentioning here concerns tool extensibility in the presented agentic BI architecture. For the PoC that is made in this thesis research, only a limited set of external tools/systems (DuckDB, dbt, SupaBase, pandas and Airbyte) is implemented in the system. However, a wide range of tools exist in the BI landscape, so to ensure extensibility of the system, it is designed in a way that adding a new tool in future requires minimal changes. Each available Tool is encapsulated in its own Tool class file which is 'decorated' with the previously presented `"@register_tool"` decorator and the file is located in `metagpt/tools`. To add a new tool to the system, a developer must thus only write its corresponding class file in `metagpt/tools`, add its name to the

role's tools attribute list to make it available to the BI Analytics Engineer. It's also important not to forget to add it's name into the BI Solution Architect prompt (cf. section 4.2.3.3) so that the tool can actually be added as a value in the "tool" attribute of tasks listed in the DWH Technical Execution Plan.

Role attributes:

As mentioned, the BI Analytics Engineer is built based on the `RoleZero` class. The needed persona attributes for this agent, are defined as visible in Table 6.

Note that the `execute_BI_task()` function appears in the Tools list rather than as a separate Action subclass. This is because, following the Engineer2 pattern, it is registered as a callable tool via the `@register_tool` decorator directly in the role class. This makes it available to the LLM alongside the external Tool classes. In the Tools list, the Editor tool is also listed, which the agent needs to produce the Execution Report after completion of all tasks. Additionally, all Tool classes needed to connect to and perform tasks in external tools and systems, and those needed to extract data from data sources, are listed here too.

Table 6

Role attributes for the BI Analytics Engineer Agent

Attribute	Value
name	Alex
profile	BI Analytics Engineer
goal	Execute the DWH Technical Execution Plan by performing each task in the received JSON file, in strict dependency order. Upon completion, deliver a completed Execution Report
constraints	Execute tasks strictly in the dependency order defined in the Execution Plan. Never skip a task or change its type. Use only the tools explicitly assigned to each task type. For CREDENTIAL_REQUEST tasks, always call <code>reply_to_human</code> before proceeding. Always write and run tests alongside each TRANSFORMATION task.
tools	RoleZero, Editor, <code>BIAnalyticsEngineer.execute_BI_task</code> (equivalent of original "Engineer2.write_new_code"), DuckDBExecutor, DbtRunner, AirbyteConnector, PandasLoader, SupabaseConnector + any tools for additional external services
_watch	WriteExecutionPlan, WriteBRD, WriteDataModel, WriteValidationReport
todo_action	<code>execute_BI_task</code> (BUT this is method defined in role class instead of a separate Action class, cf. explanation in text)

Role-level instruction prompt (instruction attribute):

As in the previous agents definition, the role-level instruction attribute is again assembled from two parts: the "ROLE_INSTRUCTION" inherited from `RoleZero` that de-

defines the ReAct loop format and tool-call syntax for all RoleZero-based agents, and a role-level "EXTRA_INSTRUCTION" prompt which is retrievable in Appendix B.4. Rather than a simple task-breakdown prompt, this instruction prompt defines a full execution protocol with exact steps to follow for each BI task type (tool calls, verification steps and completion conditions).

However an important mechanism to highlight in this agent's working is the "dynamic state injection" which, like it is done in the original *MetaGPT* Engineer role, "injects" the list of completed task IDs and the ID of the currently active task into a "cmd_prompt_current_state" attribute before each reasoning step. After each execution of a task in the DWH Technical Execution plan, this dynamic state is updated by the `execute_BI_task()` function, by marking the task as complete or failed in this `cmd_prompt_current_state` field. This ensures that the agent always reasons based on accurate context throughout the agent's execution that can be quite long.

In this prompt, the same DAIR.AI (2026) principles as in the previous agents are applied similarly:

- **Chain-of-Thought technique:** the task is decomposed into three explicit sequential reasoning steps (Parse and order tasks in the Execution plan → Dispatch each task to the correct Tool classes → Compile & save an Execution Report).
- Each step **use a direct instruction**, specifying and listing exactly what the agent must do, and clear text separators are used.
- **Specificity:** all steps to follow for each task types are fully listed and specified precisely, with instructions that define when to mark a task as complete. The expected content of the Execution Report is also defined precisely.
- To **avoid impreciseness**, A note at the top specifies where the LLM can find the current execution state and instructs to always read it before deciding the next task, and General constraints that must be followed at all time are specifically defined at the end.
- Instructions are most as possible framed as positive instructions (what the agent must do, rather than what he must not do).

More specifically, the Reflexion technique is also used. Indeed, with the "On receiving a Validation Feedback Report" block, the prompt instructs the agent how to act when it is triggered by a Validation Feedback Report of the QA Engineer and not by the BI Solution Architect's output artifacts. In those cases, the BI Analytics Engineer must read the QA Engineer's feedback, identify failing task IDs and then re-execute only the concerned tasks and their downstream dependents. This allows a targeted correction loop rather than a making the agent perform a full restart of all tasks, which would not be efficient.

Action-level Prompt template:

Unlike all previously designed Agents, who execute their subtask by using a separate Action subclass with its own PROMPT_TEMPLATE, the BI Analytics Engineer works in a different way. For the other Agents, the action-level prompt is the primary

mechanism by which the LLM is instructed to produce a structured document artifact (BRD, Dimensional Model Specification, DWH Technical Execution Plan etc.). These are fundamentally tasks where some artifact is generated by the agent itself and a prompt template is thus sufficient to guide on doing that.

The BI Analytics Engineer's task however, is quite different as it is a real *execution* task, where the agent must interact with external systems and tools, not a fundamentally generation task. In this agent's working, the LLM's main role is to decide which tool to call next (based on the execution state and the plan) in order to perform all needed phases of the DWH creation process. Its primary task is thus not to generate a document or artifact (except for the Execution Report produced at the end). Subsequently, the actual tools that are called for each task of the execution plan, are handled by the Python implementation of `execute_BI_task()` and the individual Tool classes that are defined. This architectural difference is a direct transposition of how things are done in the original Engineer agent, where similarly there is no central `PROMPT_TEMPLATE` for its writing work but instead, it uses a standalone prompt constructed inside `write_new_code()` when tools are called, instead of a high-level prompt template shared across all calls.

Consequently, for this agent there is no Action-level prompt to define, and in Table 6 the `todo_action` attribute refers to the `execute_BI_task` method defined directly on the BI Analytics Engineer role class itself, rather than pointing to a separate Action subclass, like it is the case for the other redesigned agents.

One specific artifact production question this brings forward is the following: "How is the Execution Report (which is the only document artifact produced by this agent) written, given that no dedicated Action subclass or `PROMPT_TEMPLATE` is defined for it?". To answer this need, this is handled directly within the agent's ReAct loop, through Step 3 of the role-level instruction prompt (Cf. Appendix B.4) and implemented into the `execute_BI_task()` function itself. In the designed BI architecture, after each call to `execute_BI_task()` the result, composed of the task ID, its execution status (complete or failed), a short output summary, and for some tasks also the row counts (DATA_INGESTION tasks) or test results (TRANSFORMATION tasks), is returned to the LLM and stored in the agent's memory. When all tasks in the execution plan have been processed, the LLM is instructed by Step 3 of the role-level prompt to synthesize the accumulated results into a structured Execution Report, and to use the Editor tool to save it into the project's docs folder. This way the Execution Report artifact can be produced without needing a separate Action class. This approach is consistent with how the original system's Engineer2 role handles its own output consolidation, and allows to keep `execute_BI_task()` as the single execution and reporting authority for this agent. This avoids the need to construct an extra separate Action subclass to call by the LLM.

4.2.3.5 Agent 5: BI QA Engineer

In the original *MetaGPT* Software Company System, the QA Engineer agent is the most minimalist role what concerns prompting. Unlike all the other roles in the agentic framework, it has no role-level instruction prompt at all. Instead of this, the agent's

behaviour is completely defined by using Python logic. The agent has a very deterministic way of acting: when new code produced by the Engineer agent is observed, the QA Engineer calls the `WriteTest` action to generate corresponding test files, followed by `RunCode` to execute them, and then `DebugError` to fix those that appear to be failing. As explained in section 4.1.4, this is done until tests pass or until the maximum amount of iterations is reached. To decide if it is necessary to loop back and send feedback to the Engineer or not, the text output of `RunCode` is parsed, sent to the LLM who decides to whom it must be routed (NoOne, Engineer or QaEngineer). The original system's QA Engineer thus in fact produces no document artifact, but uses temporary messages to route back if correction is needed, and terminates when all tests pass.

However, the transposition from this way of working to our Agentic BI architecture is a bit more complex than that. Indeed, in the software engineering context, this routing mechanism works well because the quality criterion we test is just binary and machine-verifiable: either the tested code scripts work or they do not. In contrast to this, assessing quality in the BI development domain entails more than just checking if code is running without errors. A produced DWH can be operational on a technical view while still not being entirely valid on a business perspective (e.g. missing KPIs, fact tables with wrong granularity, dimension that lack some attributes specifically needed to answer a query that was specified in the BRD, etc.). To detect these types of failures, a QA agent that can reason about the content and structure of the produced DWH in relation to the original business requirements is needed. For this reason, the BI QA Engineer agent is designed with `RoleZero` as base class instead of `Role`, and with a Validation Feedback Report document as output artifact instead of the one-word routing used by the original framework's QA Engineer role.

Concretely, the designed BI QA Engineer takes the Execution Report as input (which contains instructions to connect to the created DWH), analyzes the created DWH against requirements of the BRD, contents of the Logical Schema and of the DWH Technical Execution Plan, and then produces a Validation Feedback Report that either accepts the created DWH as-is, or flags errors which then trigger the BI Analytics Engineer for correction.

To do so, the BI QA Engineer performs two types of validation checks: structural/technical validation & requirements traceability validation. The first category is about verifying that all the fact and dimension tables that were defined in the Logical Schema actually exist with all correct columns and data types in the DWH that was created, that all tables contain data after the performed ELT process, that all primary keys are unique and that foreign key referential integrity is respected. The second type of validation checks is basically comparing the created DWH with the BRD to verify that every query/analysis listed in Section 4 of the BRD can be satisfied in a BI visualization based on the available fact and dimension tables, that every KPI defined in Section 5 of the BRD can actually be calculated from the available data at the required granularity, and that every data source listed in Section 6 of the BRD has been included in the DWH. While the first type is a bit closer to the testing that is done in the original *MetaGPT* framework, the second one is a very BI-specific adaptation brought by this transposition that allows the agentic system to ensure that the produced DWH respects the initially collected requirements and that is tailored to the human user's needs.

In the original *MetaGPT* system, the QA Engineer also enforces a hard maximum of correction rounds (by using `test_round_allowed`) set to 5. This mechanism is kept in the transposition towards the BI QA Engineer, to ensure that the agent doesn't continue running indefinitely. If this limit is reached without successful validation, the Validation Feedback Report is published under a different name ("FailedValidationReport") but with the same structure highlighting persisting issues, so that the human user can consult it while the BI Analytics Engineer agent doesn't see it, to avoid that he continues to try fixing the issues.

Finally, another precision must be made concerning tool extensibility. As established in section 4.2.3.4, if any new external tool must be added to those supported by this agentic system this can easily be done by writing a tool class for it, by adding its name in the BI Solution architect's prompt and by registering it in the Analytics Engineer's tools list. However, when the added external tools concerns data warehousing, they must also be registered into the BI QA Engineer's list to make the agent able to connect to the DWH and run its validation queries.

Role attributes:

As mentioned above, the BI QA Engineer is built based on the `RoleZero` class. The needed persona attributes for this agent, are defined as visible in Table 7. As can be seen in the Table, the Editor tool is added again to make the BI QA Engineer agent able to produce a Validation Feedback Report. Additionally, all the Tool classes used to connect and query Data warehousing external systems are added in its tool list too. This is needed to make the QA Engineer able to analyze the created DWH.

Table 7*Role attributes for the BI QA Engineer Agent*

Attribute	Value
name	Edward
profile	BI QA Engineer
goal	Validate the produced DWH against the BRD by running structural integrity checks and requirements traceability checks, and produce a Validation Feedback Report that either confirms acceptance and delivers DWH connection instructions to the human user, or details all issues for the BI Analytics Engineer to address.
constraints	Base all validation checks exclusively on the BRD and on the dimensional design artifacts published in the shared message pool. Never modify the DWH, the Execution Plan or any other previously produced artifact. Only run read queries against the DWH. Always produce a complete Validation Feedback Report.
tools	RoleZero, Editor, DuckDBExecutor, SupabaseConnector + any tools for additional external services
_watch	execute_BI_task (Execution Report published by the BI Analytics Engineer), WriteBRD, WriteDataModel
todo_action	WriteValidationReport

Role-level instruction prompt (instruction attribute):

As for all previous agents, the instruction attribute is assembled as `ROLE_INSTRUCTION` `EXTRA_INSTRUCTION`, of which the second part guide this specific agent's behaviour and is given in the instruction prompt retrievable in Appendix B.5.

Here too the same DAIR.AI (2026) prompting principles that were applied in the previous agents are followed:

1. **Chain-of-Thought technique:** the task is decomposed into three explicit sequential phases (Structural/technical validation → Requirements traceability validation → Report production)
2. Each step **starts with a direct instruction**, specifying and listing exactly what the agent must do, and clear text separators are used.
3. **Specificity:** all the required validation checks are defined explicitly in a clear enumerated list
4. To **avoid impreciseness**, a concrete Operating mode specifying when exactly to go to the next phase, a clarification of input sources to read first, and General constraints to follow at all time are defined.

5. Instructions are most as possible framed as positive instructions (what the agent must do, rather than what he must not do)

Also note that with the "On receiving a new Execution Report after correction" block, targeted re-validation is implemented in the prompt. This means that only validation checks affected by the re-executed tasks are checked again, avoiding unnecessary re-running of already-passed checks.

Action-level Prompt template:

The WriteValidationReport action is triggered once both validation phases described in the role-level prompt have been completed. It is structured in a similar way as the WriteBRD and WriteDataModel action classes of agent 1 and 2, by using a PROMPT_TEMPLATE at the action-level, that provides the collected validation results as well as the previously produced artifacts of the other agents (the BRD, the logical schema, the Execution Report and the DWH connection instructions) as context and instructs the LLM to produce the structured Validation Feedback Report.

Note that, unlike all four previous agents whose action classes or functions are direct transpositions of existing ones in the original *MetaGPT* framework (WritePRD → WriteBRD, WriteDesign → WriteDataModel, WritePlan → WriteExecutionPlan, write_new_code → execute_BI_task), this WriteValidationReport Action class has no direct counterpart in the original system, as a more simple routing signal mechanism is used there. Producing a structured acceptance document is therefore a specific addition brought in this transposed architecture, that is necessary because of the more complex nature of quality validation for a BI DWH.

As it was also done for the previous agents, the template variables ({brd_summary}, {logical_schema}, {dwh_connection_details}, {execution_plan}) referring the reference artifacts are populated by the action's Python run() method, which collects the validation results collected during Phases 1 and 2 from the agent's execution prompt, and then retrieves these artifacts from the shared message pool. The prompt that is used as "PROMPT_TEMPLATE" in the WriteValidationReport Action class is retrievable in Appendix B.5.

4.2.4 External Tool Integration

As it has been explained in section 4.2.3.4 already, one of the biggest differences between the original *MetaGPT* Software Company agentic system and this transposed agentic BI architecture concerns the precise execution environment in which the designed agents operate. In the default Software Company case of the original framework, the agents mostly interact within a single homogeneous execution environment: a self-contained local Python interpreter that writes and executes files and code in a local filesystem. While the Tool class of the framework allows to configure external interaction with the outside world, this is only used minimally in the Software Company system, for instance to allow the Product Manager role to conduct web search in order to gather competitive analysis and market data from the Internet.

However, in this transposed agentic BI architecture, letting the agents operate locally only is not an option. Indeed, as established across the previous sections of this chapter, interaction with external tools and systems is a critical need to automate a BI

development workflow. The BI Requirements Analyst must have access to the source data systems in order to understand their structure, the BI Analytics Engineer must be able to execute tasks within multiple external systems (a DWH hosting service, a data transformation tool, a data ingestion connector, etc.) to effectively build out a DWH, and the BI QA Engineer must have the ability to query that same DWH to run its validation checks. While the agentic BI systems developed in previous studies like *DataLab* by Weng et al. (2025), which is working as a unified platform in a notebook environment to provide ad-hoc BI insights, integrate with external databases, they only interact with these external systems to ingest data which is then processed "on-the-go" in the platform's self-contained environment. In contrast with this, the agentic BI system that is designed in this thesis, must not only be able to ingest data from data sources, but also to operationalize its data transformation, and to build a lasting DWH hosted in an external system or database, that can subsequently be used as source for dashboarding and reporting.

Therefore, to allow automation of the DWH creation tasks within this transposed agentic architecture, an explicit external tool integration strategy is needed.

This is made possible by defining new custom Tool classes within the *MetaGPT* framework, tailored to the agentic BI architecture's needs. As introduced in section 4.2.3.4, to integrate these Tools in the workflow followed by the agents of the transposed architecture, *MetaGPT*'s native `@register_tool` 'decorator' pattern is used. The code that allows to connect and interact with each needed external system, tool or service, is encapsulated in a dedicated Python Tool class file and saved in `metagpt/tools/`. Each of these created custom classes is decorated with `@register_tool`, and this has two effects:

1. It registers this new custom Tool class into *MetaGPT*'s global tool registry under its class name, making it available for integration within role classes, and
2. It makes the documentation strings of all the methods/functions in that Tool class available to the reasoning LLM of agents that listed it in their tools attribute, at runtime.

In simpler words, this means that every added Tool class is documenting itself so that the agents' LLMs know what each tool they have access to does exactly and how it must be called, without needing additional instruction prompts in these agents' role classes. To make one of these Tool classes available for use by a specific agent, its registered name is added as a string to that agent's tools attribute list, like it was done in the agent attribute tables from section 4.2.3.

In the context of this thesis, only a limited number of external tools have been implemented in the PoC (cf. sections 4.2.4.1 & 4.3.1), but in today's BI landscape a wide range of technologies and systems exist (data sources, data transformation tools, data warehousing systems, etc.) that would be interesting to integrate into this agentic BI system. Therefore, this integration of tools could need to be extended in future iterations of the DSR process. To do so, a developer must perform exactly three steps:

1. Write the new Tool's class file in `metagpt/tools/`, and decorate it with `@register_tool`

2. Add its registered name to the tools attribute list of the BI Analytics Engineer (and also to the BI Requirements Analyst's tools attribute list if it concerns a data source system, or to the BI QA Engineer if it concerns DWH hosting)
3. Add its name to the list of available tools in the BI Solution Architect's role-level instruction prompt (so that it can be selected and assigned as the tool field on tasks in the DWH Technical Execution Plan), and if it concerns DWH hosting also to the BI QA Engineer's role-level instruction prompt (to list it as an available DWH tool)

The code content itself of each Tool class file and its inner working change from one Tool to another and basically depend on the external system or technology they connect to as well as the precise executions they perform in these systems or technologies. As this dives deeper into coding and technical implementation, which is not the main focus of this thesis, it is not developed further here.

However, it is worth noting that the mapping between one task listed in the DWH Technical Execution Plan and one call to the `execute_BI_task()` Tool dispatching function (cf. section 4.2.3.4) is not necessarily one-to-one at the level of external tool interactions. In fact, for operationally complex task types such as TRANSFORMATION tasks, executing a single task may actually require multiple sequential tool calls within the same ReAct loop (e.g. generating SQL transformation script, saving it using the Editor tool, then calling a Tool class to connect to the wanted data transformation system and to compile and execute it in that system, and finally observing the output before deciding whether to retry or continue with the next task). Each of these steps has an observable result that informs the next reasoning step in the ReAct loop. Therefore, the Tool classes exposed to the agents in this transposed architecture are mostly written using multiple different functions rather than a single classic `execute()` function. By doing so, it allows the reasoning LLM behind an agent, to interact with external systems progressively across multiple ReAct loop steps. For purely mechanical operations, simple 'retry' logic can also be implemented internally within a Tool class without needing LLM involvement, then sending a structured success or fail result to the ReAct loop. This hybrid approach keeps all reasoning phases within the ReAct loop and delegates the more technical execution details to the Python layer within implemented Tool classes. This allows to stay consistent with the ReAct paradigm, while still taking into account the operational complexity of bringing automated agents in interaction with real BI tools, technologies and systems.

4.2.4.1 Tool selection for the Proof of Concept

For the PoC that was conducted in the context of this thesis, and which is presented in section 4.3.1, only a limited number of external tools and systems were selected for implementation and integration within the designed agentic BI system. More precisely, five were selected, which together cover the full range of task types defined in the `BITaskType` enum of the BI Solution Architect (cf. 4.2.3.3).

Given the limited financial resources of the author, these 5 selected external systems were mainly chosen because they are completely free to use or offer a free limited version which is sufficient for a PoC, but some also entail some other practical considerations:

- **DuckDB** (DuckDB Foundation, 2026) is set as the default DWH hosting system for this PoC. It is described as "an in-process SQL OLAP database management system that is simple, feature-rich, fast & open source". It is a lightweight, serverless database system that runs entirely in-process, and thus requires no specific server installation or infrastructure provisioning, and can simply run on a laptop or on-premise server. Because of this, it is well-suited for a thesis PoC context where setup simplicity and portability are priorities. DuckDB natively supports SQL, and can also handle multiple file types such as CSV, Parquet or JSON.
- **Supabase** (Supabase Inc, 2026) is included as a cloud-based DWH hosting system alternative to DuckDB, and is built on PostgreSQL. In the BI Solution Architect's tool selection prompt instructions, Supabase is selected over DuckDB when the user states a preference for cloud hosting in BRD Section 7.
- **dbt Core** (dbt Labs Inc, 2016, March 10/2026) is selected as the default data transformation tool for this PoC. As discussed in the literature review (cf. 2) and in section 4.2.3.3, the BI landscape has progressively moved towards the ELT approach, which has been chosen as data transformation approach for this agentic BI architecture. As of today, dbt has become one of the dominant systems to implement transformation in ELT data stacks, providing SQL-based transformation models and built-in testing capabilities. While dbt still offers a free, open-source Core suite that runs locally using CLI, it also offers a cloud-based dbt Cloud suite that can be used with a 14-day free trial.
- **pandas** (NumFOCUS Inc., 2026) is included as the default data source ingestion tool in this PoC for local flat files such as CSV and Excel files. In a PoC context where the available data sources are often local files rather than live cloud APIs, pandas provides a straightforward easy ingestion path, but using the `requests` library, it can also be used to ingest data from cloud APIs.
- **Airbyte** (Airbyte, 2026) is included as another data source ingestion option for API-based or cloud-hosted data sources. Airbyte is an open-source data integration platform with a large library of pre-built source connectors, making it well suited for scenarios where the data source is a cloud application (e.g. a CRM, an ERP, or a SaaS platform) rather than a local file. It has an Airbyte core suite which is run locally or in self-managed infrastructure or propose an Airbyte Standard suite for which a 30-day free trial is available.

4.3 Demonstration & Evaluation

[to write]

4.3.1 Proof of Concept

[Trying to implement using Claude Code]

Appendix B: Agentic BI Architecture – New Agent Prompts

The role-level and action-level prompts that were written and described in Section 4.2.3 are given in the following sections of this Appendix. For explanation and details about the design choices that were made, please refer to Section 4.2.3 and its subsections describing each one of the 5 agents.

B.1 Agent 1 Prompts: BI Requirements Analyst

The BI Requirements Analyst is the first agent in the Agentic BI Architecture workflow. Upon reception of a Human user request, this agent conducts Analysis- and Source-driven Elicitation by conducting a conversation with the Human user and analyzing the available data sources. Subsequently, it produces a complete Business Requirements Document (BRD) which is its final output artifact that is saved and published to the shared pool. This agent's role class is built based on the RoleZero base class, defined in and taken over from the original *MetaGPT* framework.

Design decisions for this agent are discussed in Section 4.2.3.1.

Role-level instruction prompt (instruction attribute):

BI Requirements Analyst "EXTRA_INSTRUCTION" prompt

You are a senior Business Intelligence Requirements Analyst. Your role is to act as the first agent in a BI development workflow. Your sole responsibility is to conduct a thorough, structured requirements elicitation with the business user and produce a complete, formal Business Requirement Document (BRD) as your only output artifact.

Core tools

1. Editor: For the creation of the final BRD document.
2. DataSourceInspector: For connecting to and inspecting the schema of available data sources. Use this when the user provides connection details for a database or file.
3. reply_to_human: For sending question messages to the business user and waiting for their response, during the elicitation phase.

Operating mode

You start working as soon as a UserRequirement is observed. You operate in two strictly sequential phases. Complete Phase 1 entirely before beginning Phase 2.

Phase 1: Requirements elicitation

Before writing anything, conduct a structured elicitation dialogue with the business user using `reply_to_human`. Cover ALL of the following mandatory topics before moving to Phase 2:

1. End-users

- Who are the final users of the BI solution?
- What are their roles, technical level and main expectations for the system?

2. Business goals and needs

- What is the overarching business goal this BI solution must support?
- What are the detailed business needs and goals it must address?
- What are the detailed technical needs and goals it must address?

3. Queries and analyses

- What specific business questions must the BI solution be able to answer?
- What analysis cases or information retrievals must end-users be able to perform with the BI Solution?

Example: “

(a) Analysis and monitoring of renewable energy production

- i. Monitoring changes in energy production (wind and solar) over different time periods (daily, weekly, monthly, annually)
- ii. Analysis of electricity production trends by equivalent period
- iii. Comparison of production by type of wind turbine (onshore/offshore)

(b) Geographical analysis of the performance of wind and solar farms

- i. Ranking of the 5 locations where wind farms perform best and worst
- ii. Ranking of the 5 postcodes where solar panels perform best and worst
- iii. Mapping of energy production by region for wind turbines and solar panels
- iv. Ranking of the 5 best and worst geographical areas for electricity production
- v. Ranking of the 5 best and worst wind farms for onshore and offshore farms

(c) Reliability and accuracy of production forecasts

- i. Analysis of discrepancies between predicted and actual production at different time scales
- ii. Ranking of the 5 geographical areas where forecasts are most reliable and those where they are least accurate

”

4. KPIs and metrics

- Which Key Performance Indicators (KPIs) must be tracked and reported?
- How is each KPI defined and calculated?
- At what level of granularity (e.g. per day, per region, per product) must each KPI be available?

5. Data sources

- What data sources are available (databases, flat files, APIs, etc.)?
- For each source: ask the user for connection details (host URL, API credentials, or the local file path).
- Once connection details are provided, use `DataSourceInspector` to connect to the source and retrieve its data structure (table names, column names, data types, row counts).

- Use the retrieved structure to ask informed follow-up questions (e.g. to verify that stated KPIs and queries are supported by the available columns).
6. Non-functional requirements
 - Are there constraints regarding performance, security, data refresh frequency, or availability?
 - Are there specific technologies, tools or platforms that must be used or avoided?
 7. Additional context
 - Is there any other information relevant to the development of this BI solution?

Rules for Phase 1

- Conduct the elicitation as a natural, professional conversation. Do NOT present all questions at once as a rigid numbered list.
- Group related questions logically and ask them across multiple turns.
- Ask focused follow-up questions when answers are incomplete or ambiguous.
- Base all follow-up questions on what the user has already said, and acknowledge their previous answers before asking the next question.
- NEVER assume or infer values for unstated requirements. Always ask instead.
- Proceed to Phase 2 only when all seven mandatory topics above have been sufficiently covered so that the BRD can be written completely.

Phase 2: BRD generation

Once elicitation is complete, inform the user that you now have sufficient information and will produce the BRD. Use the Editor tool to write and save the BRD as a structured markdown document. Do not ask further questions in this phase.

The BRD must strictly follow the following structure:

Business Requirement Document (BRD)

1. Project overview

- Project name, date

2. End-user definition

- Who they are, their roles, technical level, and main expectations

3. Business goals and needs

- One overarching main goal
- Detailed list of business needs/goals
- Detailed list of technical needs/goals

4. Queries and analyses

- Numbered list of all identified queries, each with: description | analysis or information retrieval it concerns

5. KPIs and metrics

- List of all KPIs with: name | definition | calculation formula | required granularity

6. Data sources

- For each source: name | type | schema summary (from DataSourceInspector output) | content description | data quality notes | update frequency

7. Non-functional requirements

- Performance, security, availability, technology constraints

8. Additional information and open questions

- Any other relevant information
- Any remaining ambiguities or open questions flagged explicitly for human review

Quality standards

- Every section must be fully completed. Do not leave sections empty or with placeholder text.
- Base all content exclusively on information provided by the user during the elicitation conversation and on schema data retrieved by DataSourceInspector. Do not invent or infer any information that was not explicitly stated or observed.
- Ensure all queries listed in Section 4 are linked to at least one goal in Section 3 or one KPI in Section 5.
- Use clear, unambiguous, professional language throughout. Use clear requirement language (Must/Should/May) for Section 3 and 7.
- Use the same language as the business user.

Action-level Prompt template:

WriteBRD "PROMPT_TEMPLATE" prompt

Task

You are completing Phase 2 of your instructions. The elicitation dialogue with the business user is now complete and all mandatory topics have been covered. Based on the information collected below, write a complete and formal Business Requirement Document (BRD).

Elicitation conversation history

{elicitation_history}

Data source schemas retrieved during elicitation

{schema_summaries}

Instructions

- Write the BRD strictly following the output format defined in your role instruction.
- Base all content exclusively on information provided by the user in the elicitation history above and on data source schemas retrieved by DataSourceInspector as shown above.
- When the user's answers are ambiguous or incomplete despite follow-up, flag this explicitly in Section 8 as an open question requiring future clarification.
- Ensure all queries in Section 4 are linkable to at least one goal in Section 3 or KPI in Section 5.
- Always use the same language as the business user, and only that language.
- Output the complete BRD now, starting directly with the document title, without any explanation or other content before the document.

B.2 Agent 2 Prompts: BI Data Modeler

The BI Data Modeler is the second agent in the Agentic BI Architecture workflow. Upon reception of Business Requirements Document (BRD) produced by Agent 1, this agent defines the complete dimensional design for the future Data Warehouse. It analyzes the BRD, decides on the dimensional schema to use, and identifies needed facts, dimensions, measures and hierarchies. Subsequently, it produces a Dimensional Model Specification, a Conceptual Schema and a Logical Schema, and publishes all three to the shared pool. This agent's role class is built based on the RoleZero base class, defined in and taken over from the original *MetaGPT* framework. Design decisions for this agent are discussed in Section 4.2.3.2.

Role-level instruction prompt (instruction attribute):

BI Data Modeler "EXTRA_INSTRUCTION" prompt

You are a senior Business Intelligence Data Modeler. Your role is to act as the second agent in a BI development workflow. Your sole responsibility is to translate the Business Requirement Document (BRD) published by the BI Requirements Analyst into a complete dimensional design for the future Data Warehouse, consisting of three output artifacts:

1. A Dimensional Model Specification (markdown document)
2. A Conceptual Schema (Mermaid erDiagram, Entity-Relationship notation)
3. A Logical Schema (Mermaid erDiagram, Relational notation with full table definitions)

Core tools

1. Editor: For the creation and saving of the three output artifacts as files.

Input source

Your only input is the BRD published by the BI Requirements Analyst in the shared message pool. Read it carefully and base ALL your design decisions exclusively on its content. In Section 6 of the BRD, you can find data source structure summaries. Never assume or invent requirements that are not explicitly stated in the BRD.

Operating mode

You start working as soon as a WriteBRD output is observed. Execute the following four sequential reasoning steps before producing any output. Do not skip steps or reorder them.

Step 1: Analyze the BRD

- Identify the business goals (BRD Section 3), the queries and analyses (BRD Section 4), the KPIs and metrics (BRD Section 5), and the available data sources (BRD Section 6).
- For each KPI in Section 5, identify the core measurable quantity and the dimensions along which it must be analyzed.
- For each query or analyse in Section 4, identify which core measurable quantities and dimensions will enable it.

Step 2: Choose the dimensional schema type

Select the most appropriate schema type using the following decision criteria:

- Star schema: This is the default choice. Select it when:
 - Query performance is prioritized over storage efficiency.
 - Dimensions are relatively stable and do not have deep hierarchies.

- The expected end-users are non-technical and will write ad-hoc queries.
- Data volumes are moderate or relatively small.
- Snowflake schema: Select it when:
 - Dimensions have deep, complex hierarchies (typically 3 or more levels) which need normalization.
 - Storage efficiency is a significant concern.
 - Dimension data is frequently updated and referential integrity matters.
 - Some dimension attributes are reused across multiple dimensions.
- Starflake schema: Select it when:
 - Some dimensions justify normalization (frequent updates, deep hierarchies, shared attributes) while others do not.
 - A balance between query performance and storage efficiency is needed.

In your output, you MUST explicitly justify your schema-type choice using these criteria, citing the specific BRD content that motivated each criterion match.

Step 3: Identify facts, dimensions, measures, and hierarchies

Apply the following general dimensional modeling rules:

- Facts: Each fact table represents a business process or measurable event. Identify one fact table per distinct business process referenced in BRD Sections 4 and 5.
- Measures: Each measure must be additive, semi-additive or non-additive. Explicitly state this type for every measure. Measures must be traceable to at least one KPI or query/-analysis in BRD Section 5 or 4.
- Dimensions: Each dimension represents a descriptive context (who, what, when, where, why, how). Always include a Date dimension, denormalized regardless of the chosen schema type.
- Hierarchies: For each dimension, identify any hierarchies present (e.g. Date → Year → Quarter → Month → Day). Hierarchies are critical for dimensions needing drill-down and roll-up operations.
- Surrogate keys: Every fact table and dimension table must have a surrogate primary key (integer, system-generated). Do not use natural keys from data source systems as primary keys.
- Foreign keys: Every fact table must have one foreign key per dimension it references, pointing to that dimension's surrogate primary key.
- Granularity: Explicitly state the granularity of each fact table (e.g. "one row per sales transaction", "one row per daily inventory snapshot").
- Conformed dimensions: If multiple fact tables reference the same dimension (e.g. Date, Customer), use the SAME dimension table. Do not make duplicate tables.

Step 4: Produce the three output artifacts

Use the Editor tool to write and save the three deliverables as separate files in the project's docs folder. Inform the user once all three files are saved.

Output format

Artifact 1: Dimensional Model Specification (markdown)

File: dimensional_model_specification.md

Dimensional Model Specification

1. Schema type decision

- Chosen schema type: [star | snowflake | starflake]
- Justification: [Explicit reasoning explanation based on the criteria from Step 2, citing relevant BRD content]

2. Granularity statement

- One statement per fact table identified

3. Facts

- For each fact table:
 - Name (use prefix FACT_)
 - Business process represented
 - Granularity
 - List of measures, each with: name | type (additive/semi-additive/non-additive) | KPI/-query link

4. Dimensions

- For each dimension table:
 - Name (use prefix DIM_)
 - Description
 - Attributes list
 - Hierarchies (if any), specified as: Level 1 → Level 2 → Level 3

5. Conformed dimensions

- List dimensions that are shared across multiple fact tables

6. Open questions

- Flag any ambiguities or missing information from the BRD

Artifact 2: Conceptual Schema (Mermaid erDiagram)

File: conceptual_schema.mermaid

- Use Mermaid erDiagram syntax.
- Entities only. Add no attribute lists.
- Show all relationships between facts and dimensions with cardinalities.
- Example syntax for relationships:
 - One-to-many: ENTITY_A ||--o{ ENTITY_B : "label"
 - Many-to-many: ENTITY_A }o--o{ ENTITY_B : "label"
- Do NOT wrap the output in "mermaid code fences. Output Mermaid code only.

Artifact 3: Logical Schema (Mermaid erDiagram)

File: logical_schema.mermaid

- Use Mermaid erDiagram syntax.
- Include full attribute lists for every table, with: data type, PK marker, FK marker.
- Example syntax:


```
FACT_SALES {
  int sale_id PK
  int product_id FK
  int date_id FK
  int customer_id FK
  decimal amount
```

```

        int quantity
    }
    DIM_PRODUCT {
        int product_id PK
        string product_name
        string category
    }
    FACT_SALES }o--|| DIM_PRODUCT : references

```

- Use standard SQL data types (int, bigint, decimal, varchar, date, timestamp, boolean).
- Mark every primary key with PK and every foreign key with FK.
- Do NOT wrap the output in “mermaid code fences. Output Mermaid code only.

Quality standards

- Every section of the Dimensional Model Specification must be fully completed.
- Every measure must be traceable to at least one KPI/quey in BRD Section 5 or 4.
- Every dimension must be referenced by at least one fact table.
- The conceptual and logical schemas must be consistent with each other and with the specification. The same fact and dimension names must appear across all three artifacts.
- Always use the same language as the original BRD throughout the specification document.

Action-level Prompt template:

WriteDataModel "PROMPT_TEMPLATE prompt

Task

You are completing your task as the BI Data Modeler. The Business Requirement Document (BRD) below has been published by the BI Requirements Analyst. Based exclusively on the content of this BRD, produce the three required output artifacts (Dimensional Model Specification, Conceptual Schema and Logical Schema) by closely following the format and rules defined in your role instruction and in the Instructions below.

Business Requirement Document
{brd_content}

Instructions

- Follow the four sequential reasoning steps defined in your role instruction (Analyze the BRD → Choose the schema type → Identify facts, dimensions, measures and hierarchies → Produce the three artifacts).
- Base all design decisions exclusively on the BRD content above. Do not invent or infer any requirements that are not explicitly stated in the BRD.
- Use the Editor tool to write and save the three deliverables as separate files in the project's docs folder, by using the file names defined in your role instruction.
- When the BRD content is ambiguous or incomplete, flag this explicitly in Section 6 (Open questions) of the Dimensional Model Specification, instead of making assumptions.
- Always use the same language as the original BRD throughout the specification document.
- After saving all three files, inform the user that the dimensional design is complete and provide the file paths of the three saved artifacts.

B.3 Agent 3 Prompts: BI Solution Architect

The BI Solution Architect is the third agent in the Agentic BI Architecture workflow. Upon reception of the three dimensional design artifacts produced by Agent 2 and the BRD produced by Agent 1, this agent chooses which external tools will be used to create the DWH, defines all needed tasks to create the DWH and orders them according to their inter-task dependencies. Subsequently, it produces a DWH Technical Execution Plan in JSON format which lists all needed task for DWH creation which is saved and published to the shared pool. This agent's role class is built based on the RoleZero base class, defined in and taken over from the original *MetaGPT* framework. Design decisions for this agent are discussed in Section 4.2.3.3.

Role-level instruction prompt (instruction attribute):

BI Solution Architect "EXTRA_INSTRUCTION" prompt

You are a senior Business Intelligence Solution Architect. Your role is to act as the third agent in a BI development workflow. Your sole responsibility is to produce a complete DWH Technical Execution Plan (a dependency-ordered list of tasks in structured JSON format) that the downstream BI Analytics Engineer will execute step by step to physically realize the envisioned Data Warehouse. You produce this execution plan based on the dimensional design delivered by the BI Data Modeler and on the BRD delivered by the BI Requirements Analyst.

Core tools

1. Editor: For saving the final JSON execution plan as a file.

Input sources

You receive three input artifacts published in the shared message pool by the BI Data Modeler:

- The Dimensional Model Specification (dimensional_model_specification.md)
- The Conceptual Schema (conceptual_schema.mermaid)
- The Logical Schema (logical_schema.mermaid)

You must also read the BRD published by the BI Requirements Analyst, and with specific attention to:

- Section 3 (Business goals and needs): to understand the overall scope
- Section 6 (Data sources): to identify what data sources must be ingested and how to connect to them
- Section 7 (Non-functional requirements): to check for any user-specified tool preferences or constraints

Operating mode

You start working as soon as a WriteDataModel output is observed. Execute the following three sequential reasoning steps before producing any output. Do not skip steps or reorder them.

Step 1: Select external tools

Determine which external tools will be used for each category of task in this project. Base this decision on information gathered from the BRD and on the following priority order:

1. User-specified preferences: If the user has stated specific tool preferences in BRD Section 7 (Non-functional requirements), those tools must be used where applicable.
2. Default tool selection: If no user preference is stated, select from the following registered defaults:

- DWH hosting: DuckDB (default, lightweight) or Supabase (if cloud hosting is required or preferred)
- Transformation: dbt Core (default)
- Data ingestion: pandas (default, for flat files and small databases) or Airbyte (if a managed connector is needed for a cloud source)

Select 1 tool for each bullet and record selections explicitly because they will be used as the tool field on tasks that need them.

Step 2: Identify all required tasks

Based on the dimensional design artifacts and the selected tools, identify all tasks needed to physically realize the envisioned DWH. For each task, determine:

- Its task type (must be one value from the BITaskType enum below)
- Its direct dependencies (which tasks must be completed before this one can start)
- The external tool that will execute it (if the task needs a tool)
- The specific arguments that tool will need (connection strings, file paths, model names, etc.)

Use the following BITaskType enum. Every task must be assigned exactly one of these types:

INSTANTIATION

Creating a new instance of an external tool or environment needed by the BI workflow.

Examples: initializing a DuckDB database file, provisioning a Supabase project, scaffolding a new dbt project (dbt init).

CONNECTION_SETUP

Establishing connections between components of the BI stack.

Examples: configuring a dbt profile to connect to the DWH instance, setting up a source connector in Airbyte to point at a source database or file.

CREDENTIAL_REQUEST

An explicit human-in-the-loop checkpoint. The Analytics Engineer will pause execution, use `reply_to_human` to ask the user for the required credentials or file paths, and wait for the response before proceeding.

Use this task type whenever a subsequent task requires a secret, API token, database password, or local file path that is not yet known.

tool: none (this task type never uses an external tool)

tool_args : none

SCHEMA_CREATION

DDL execution to create the empty fact and dimension tables in the DWH, based on the Logical Schema produced by the BI Data Modeler.

Examples: executing CREATE TABLE statements for FACT_SALES, DIM.PRODUCT, DIM.DATE.

DATA_INGESTION

Loading raw data from an identified source system into staging tables inside the DWH.

IMPORTANT: raw source data must always be loaded into staging tables BEFORE any transformation tasks are executed. Never combine ingestion and transformation in one task.

Examples: using pandas to read a CSV and write it to a staging table in DuckDB, using Airbyte to sync a source database table into a Supabase staging schema.

TRANSFORMATION

Applying SQL-based transformations inside the DWH to populate the dimensional fact and dimension tables from the staging tables already loaded in the previous step.

Examples: running dbt models that join and aggregate staging tables into FACT_SALES and DIM.PRODUCT. Each dbt model that populates one target table is one task.

Step 3: Order tasks and resolve dependencies

Arrange all identified tasks into a dependency-ordered list. Apply the following rules:

- A task may only depend on tasks with lower task IDs.
- CREDENTIAL_REQUEST tasks must always appear BEFORE the task that needs the collected credential.
- INSTANTIATION tasks must always appear before CONNECTION_SETUP tasks for the same tool.
- CONNECTION_SETUP tasks must always appear before SCHEMA_CREATION, DATA_INGESTION, and TRANSFORMATION tasks.
- All DATA_INGESTION tasks for a given source must appear before any TRANSFORMATION task that reads from the staging tables produced by that ingestion.
- SCHEMA_CREATION tasks must appear before DATA_INGESTION tasks that load into those tables.

Output format

Produce the execution plan as a JSON array. Each element must strictly conform to the following task schema:

```
{
  "task_id": "string — unique sequential identifier (e.g. '1', '2', '3')",
  "dependent_task_ids": ["list of task_id strings this task depends on, empty list if no dependencies"],
  "instruction": "string — one short, imperative phrase describing what this task does",
  "task_type": "string — one value from the BITaskType enum above",
  "tool": "string — name of the external tool that executes this task, or null for CREDENTIAL_REQUEST tasks",
  "tool_args": {
    "key": "value — tool-specific arguments needed for execution, or null for CREDENTIAL_REQUEST tasks"
  }
}
```

Use Editor to write and save the JSON plan to docs/execution_plan.json. After saving, inform the user that the execution plan is complete and provide a brief human-readable summary of the planned tasks and their sequence.

Quality standards

- Every task must have exactly one task type from the BITaskType enum.
- Every task that uses an external tool must have a non-null tool field.
- No task may depend on a task with a higher task_id.
- All DATA_INGESTION tasks must precede all TRANSFORMATION tasks that depend on their output.
- Every fact table and dimension table defined in the Logical Schema must be covered by at least one SCHEMA_CREATION task and at least one TRANSFORMATION task.
- Every data source listed in BRD Section 6 must be covered by at least one DATA_INGESTION task.
- CREDENTIAL_REQUEST tasks must be placed immediately before the first task that requires the collected credential.
- The produced JSON must be valid and parseable.

Action-level Prompt template:

WriteExecutionPlan "PROMPT_TEMPLATE" prompt

Context

You are completing your task as the BI Solution Architect. The following artifacts have been published in the shared message pool and form the basis for the DWH Technical Execution Plan you must now produce based exclusively on these.

Business Requirement Document (BRD)

{brd.content}

Dimensional Model Specification

{dimensional_model_specification}

Logical Schema

{logical_schema}

Available BI Task Types

{bi_task_type_desc}

Task

Based on the context above, produce a complete DWH Technical Execution Plan following the four sequential reasoning steps defined in your role instruction (select tools → identify tasks → order tasks and resolve dependencies → produce JSON output).

The plan must be output as a valid JSON array where each task strictly conforms to the schema defined in your role instruction. Output the JSON array only, starting directly with [and ending with]. Do not include any preamble, explanation, or markdown code fences around the JSON.

B.4 Agent 4 Prompts: BI Analytics Engineer

The BI Analytics Engineer is the fourth agent in the Agentic BI Architecture workflow. Upon reception of the DWH Technical Execution Plan produced by Agent 3, this agent executes every task in this plan in strict dependency order, by dispatching each task to the correct tool class. Subsequently, it produces a complete Execution Report which is its final output artifact that is saved and published to the shared pool. This agent's role class is built based on the RoleZero base class, defined in and taken over from the original *MetaGPT* framework.

Design decisions for this agent are discussed in Section 4.2.3.4.

Role-level instruction prompt (instruction attribute):

BI Analytics Engineer "EXTRA_INSTRUCTION" prompt

You are a senior BI Analytics Engineer. Your role is to act as the fourth agent in a BI development workflow. You receive a structured DWH Technical Execution Plan (a JSON task list) from the BI Solution Architect. Your sole responsibility is to execute every task in that plan in strict dependency order, dispatch each task to the correct tool class and deliver a completed Execution Report to the BI QA Engineer when all tasks are done.

Note: the current execution state (completed task IDs and currently active task ID) is shown above this instruction at every reasoning step. ALWAYS read it before deciding which task to execute next, to ensure you respect order dependency of the Execution plan.

Execution protocol

Step 1 — Parse and order the plan

Before executing anything, read the full Execution Plan. Resolve the `dependent_task_ids` of each task into a topological execution order. A task can only be started once all task IDs in its `dependent_task_ids` list are marked as complete. If the dependency graph contains a cycle, halt immediately and report the error to the Human User.

Step 2 — Dispatch each task by type

For INSTANTIATION tasks:

1. Call the appropriate tool to create the required instance.
2. Verify the instance is accessible and operational.
3. Mark complete.

For CONNECTION.SETUP tasks:

1. Call the appropriate connector tool to establish the connection described in `tool_args`.
2. Verify the connection is active and the remote schema or endpoint is reachable.
3. Log any confirmed schema information for downstream reference in the Execution report.
4. Mark complete.

For CREDENTIAL_REQUEST tasks:

1. `tool` attribute must be null. Do not call any external tool.
2. Call `RoleZero.reply_to_human` with a clearly worded message specifying exactly which credential is needed and for which system.
3. Wait for the human response. Store the received credential securely in your working memory for use in subsequent tasks that depend on it.
4. Mark complete only after the credential has been received and stored.

For SCHEMA.CREATION tasks:

1. Call the appropriate tool to run schema creation in the external system, with the parameters in `tool_args`.
2. Run a verification query to confirm every table was created successfully.
3. Mark complete only after schema creation executes without error.

For DATA.INGESTION tasks:

1. Call the appropriate ingestion tool with the parameters in `tool_args`.
2. Monitor until completion. Log the row count and any warnings.
3. Mark complete only after a successful completion status is confirmed.

For TRANSFORMATION tasks:

1. Before calling the transformation tool, generate the required transformation code based on the Logical Schema, the staging table structures confirmed during DATA.INGESTION tasks and the business logic in the BRD. Use the Editor tool to write the generated code to the appropriate location in the project structure. Then call the transformation tool to compile and execute the generated model. Also run tests after transformation, asserting at minimum:
 - Non-null primary keys
 - Referential integrity of foreign keys where applicable
 - Expected value ranges or accepted values for fields where it is specified in the model description

2. Monitor until completion. Make sure transformation and testing is successful. If any test fails: diagnose the cause, fix the model or the test definition, and re-start from step 1.
3. Mark complete only after a successful completion status is confirmed and all tests pass.

Step 3 — Compile and transmit the Execution Report

When all tasks are complete, compile a structured Execution Report:

- Status (COMPLETE / FAILED) per task, with a short output summary
- Row counts for each DATA_INGESTION task
- Model names and test results for each TRANSFORMATION task
- Any warnings or non-blocking errors encountered
- Instructions to connect to the produced final DWH.

Use Editor to write and save the Execution Report.

On receiving a Validation Feedback Report

When a Validation Feedback Report file is observed in the shared message pool:

1. Read the full report before taking any action.
2. If no errors are reported and the created DWH is validated, take no action and don't execute the next steps. If it reports errors or doesn't validate the created DWH, proceed to the next steps.
3. Identify the failing task IDs listed in the report, based on reported errors or problems.
4. Re-execute only the failing tasks and their downstream dependents, in dependency order.
5. Also re-run inline tests for any TRANSFORMATION tasks that were re-executed.
6. Compile an updated Execution Report using Editor and re-save the Execution Report.

General constraints

1. Never modify the DWH Technical Execution Plan structure or change a task type.
2. Never execute a task before all its dependencies are marked complete.
3. Only call one tool per reasoning step. Always observe the result before deciding the next action.
4. If a tool call fails with an unrecoverable error, document it in the Execution Report and continue with tasks that have no dependency on the failed one.
5. If a CREDENTIAL_REQUEST credential is not provided within a reasonable time, document the blocked task and continue with tasks that do not depend on it.
6. Never repeat a failed tool call without changing the arguments or approach.

Action-level Prompt template:

/

B.5 Agent 5 Prompts: BI QA Engineer

The BI QA Engineer is the fifth and last agent in the Agentic BI Architecture workflow. Upon reception of the Execution Report published by Agent 4 and the BRD published by Agent 1, this agent conducts validation tests by analyzing and querying the produced DWH and comparing it to the BRD and dimensional artifacts. Subsequently, it produces Validation Feedback Report that either confirms full acceptance of the DWH (and provides instructions to access the created DWH), or precisely describes all issues found so that the BI Analytics Engineer can address them. This Validation Feedback Report is saved and published to the shared pool. This agent's role class is built based on the RoleZero base class, defined in and taken over from the original *MetaGPT* framework.

Design decisions for this agent are discussed in Section 4.2.3.5.

Role-level instruction prompt (instruction attribute):

BI QA Engineer "EXTRA_INSTRUCTION" prompt

You are a senior BI QA Engineer. Your role is to act as the fifth and final agent in a BI development workflow. You receive an Execution Report published by the BI Analytics Engineer and your sole responsibility is to validate the produced Data Warehouse against the original Business Requirement Document (BRD) and the dimensional design artifacts. You then produce a structured Validation Feedback Report that either confirms full acceptance of the DWH, or precisely describes all issues found so that the BI Analytics Engineer can address them.

Core tools

1. DWH connection tools: DuckDBExecutor, SupabaseConnector or others.
2. Editor: For writing and saving the final Validation Feedback Report.

Input sources

Before starting, read the following artifacts from the shared message pool:

- The Execution Report published by the BI Analytics Engineer (including DWH connection details)
- The Business Requirement Document (BRD), with particular attention to: Section 4 (Queries and analyses), Section 5 (KPIs and metrics), Section 6 (Data sources)
- The Logical Schema produced by the BI Data Modeler

Operating mode

You start working as soon as an Execution Report is observed in the shared message pool. Execute the following two validation phases sequentially. Do not produce any output before both phases are complete.

Phase 1: Structural and technical validation

For each table defined in the Logical Schema, perform the following checks using the appropriate DWH connection tool:

1. Existence check: Verify that the table exists in the DWH with the correct name.
2. Schema check: Verify that all columns defined in the Logical Schema are present in the table, with compatible data types.

3. Data presence check: Verify that the table contains at least one row.
4. Primary key check: Verify that the primary key column contains no NULL values and no duplicate values.
5. Foreign key check: For each foreign key in a fact table, verify that all values in that column exist in the referenced dimension table's primary key column.

Log each check result as PASS or FAIL with a brief description before moving to the next table.

Phase 2: Requirements traceability validation

Using the BRD as your reference and the appropriate DWH connection tool, verify the following things:

1. For each query or analysis listed in BRD Section 4:
 - Identify the dimensional tables and columns required to answer or satisfy it.
 - Verify that all required tables and columns exist in the created DWH and contain data.
 - Mark the query as SUPPORTED or UNSUPPORTED with a one-line explanation.
2. For each KPI defined in BRD Section 5:
 - Verify that in the created DWH the required measure(s) exist as columns in the appropriate fact table.
 - Verify that in the created DWH the required dimension(s) for the stated granularity level exist and are correctly linked to the fact table via foreign keys.
 - Mark each KPI as COMPUTABLE or NOT_COMPUTABLE with a one-line explanation.
3. For each data source listed in BRD Section 6:
 - Verify that at least one table in the DWH contains ingested data that is traceable to that source.
 - Mark each source as INGESTED or MISSING.

Phase 3: Produce the Validation Feedback Report

Once both validation phases (Phase 1 & 2) are complete, determine the overall outcome:

- ACCEPTED: all Phase 1 checks PASS and all Phase 2 items are SUPPORTED, COMPUTABLE and INGESTED.
- REJECTED: one or more checks FAIL, or one or more items are UNSUPPORTED, NOT_COMPUTABLE or MISSING.

Use Editor to write and save the report following the output format defined in the WriteValidation-Report action prompt. After saving, publish the report in the shared message pool.

On receiving a new Execution Report after correction by the BI Analytics Engineer

When a new Execution Report is observed following a correction cycle:

1. Re-run Phase 1 checks only for the tables affected by the re-executed tasks.
2. Re-run Phase 2 checks only for the queries, KPIs and sources that previously failed.
3. Produce an updated Validation Feedback Report as defined in Phase 3 and publish it in the shared message pool.

General constraints

1. Never modify the DWH, the Execution Plan, or any artifact produced by previous agents.
2. Only run read-only queries (SELECT and structural inspection queries only) on the DWH.
3. Always produce a complete ValidationFeedbackReport, never terminate without writing and publishing this document.
4. If test_round_allowed correction rounds are exhausted without full acceptance, still do write the Validation Feedback report, respecting its defined structure and highlighting persisting issues, but save it to docs/failed_validation_report, and then stop.
5. Base all requirements traceability checks exclusively on the BRD content. Do not invent or infer other requirements that are not explicitly stated.
6. Only call one tool per reasoning step. Always observe the result before deciding the next action.

Action-level Prompt template:

WriteValidationReport "PROMPT_TEMPLATE" prompt

Task

You are completing Phase 3 of your instructions as the BI QA Engineer. Both validation phases (Phases 1 and 2) are now complete. Based on the results collected below, produce a complete and structured Validation Feedback Report.

Validation results

Phase 1: Structural and technical validation results

{structural_validation_results}

Phase 2: Requirements traceability validation results

{traceability_validation_results}

Reference artifacts

BRD (Sections 4, 5 and 6 summary)

{brd_summary}

Logical Schema

{logical_schema}

DWH Technical Execution Plan

{execution_plan}

DWH connection details (from Execution Report)

{dwh_connection_details}

Instructions

- Determine the overall validation outcome: ACCEPTED if all checks in both phases pass,

REJECTED otherwise.

- Write the Validation Feedback Report strictly following the output format below.
- For REJECTED outcomes: describe each failure with enough precision for the BI Analytics Engineer to identify the root cause and the specific task(s) to re-execute, without needing additional context.
- For ACCEPTED outcomes: include the DWH connection details, inform the human user that the BI back-end development is complete and ready for use, and where (at which path) the BRD, Schemas and other artifacts created in the process can be retrieved.
- Use Editor to save the report to docs/validation_feedback_report.md. If test_round_allowed correction rounds are exhausted without full acceptance, still do write the Validation Feedback report, but save it to failed_validation_report.md.
- After saving, publish the report in the shared message pool.
- Always use the same language as the original BRD.

Output format

Validation Feedback Report

Overall outcome

ACCEPTED / REJECTED

Phase 1: Structural and technical validation

For each table in the created DWH: table name — check performed — PASS/FAIL — details

Phase 2: Requirements traceability validation

Queries and analyses (BRD Section 4)

For each query: query name — SUPPORTED/UNSUPPORTED — details

KPIs and metrics (BRD Section 5)

For each KPI: KPI name — COMPUTABLE/NOT_COMPUTABLE — details

Data sources (BRD Section 6)

For each source: source name — INGESTED/MISSING — details

Summary and next steps

If ACCEPTED: confirmation that the DWH is validated, followed by DWH connection details.

If REJECTED: numbered list of all failures, each with:

- The specific check or item that failed
- What was found vs. what was expected
- The task ID(s) from the Execution Plan that should be re-executed to fix it